AI-DRIVEN RECRUITMENT ECOSYSTEM FOR INTELLIGENT JOB MATCHING AND PREDICTIVE CAREER DEVELOPMENT

COMMON INTEGRATED SOLUTION REPORT

Team member	Student ID	Individual component
D. T. D. Perera	IT22089236	Component 1: Automated Semantic Resume Screening and IT Job Role Classification
Yenura S. Karunanayaka	IT22306272	Component 2: Autonomous Technical Interview Evaluation
K. G. Savinda N. Perera	IT22027610	Component 3: Candidate Suitability Scoring and Listwise Ranking Component 4: Predictive Skill Gap Diagnostics and Career Pathfinding

Report submitted in partial fulfilment of the requirements for the

Bachelor of Science (Honours) in Information Technology

Specialising in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

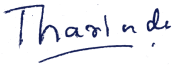
Document Publication Portal: https://r26-it-148-website.yenura02.workers.dev/

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. We declare that this common report is our own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of our knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. We grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this report, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. We have disclosed all use of generative AI and other automated tools in accordance with the module policy. We remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this report.

Student name	Student ID	Signature	Date
D. T. D. Perera	IT22089236		10 October 2026
Yenura S. Karunanayaka	IT22306272		10 October 2026
K. G. Savinda N. Perera	IT22027610		10 October 2026

Supervisor certification. The above candidates carried out this research project under my supervision.

Role and name	Signature	Date
Supervisor: Dr. Pradeep Kalansooriya		
Co-supervisor: Ms. K. V. S. Dilrukshi		

ABSTRACT

Modern corporate recruitment funnels are paralyzed by high candidate application volumes, extreme specialization, and severe information asymmetry. Legacy Applicant Tracking Systems depend on brittle keyword filters that prematurely eliminate qualified non-keyword candidates while rewarding opportunistic keyword stuffing. Furthermore, conventional hiring pipelines operate as fragmented, terminating funnels: candidates transition through disconnected tools, disparate scores are aggregated without scientific calibration, and rejected applicants receive uninformative rejection emails devoid of actionable diagnostic guidance.

This Common Integrated Solution Report documents **RecruitAI** (Research Project Identifier: **R26-IT-148**), an end-to-end, multi-modal, artificial intelligence-driven technical recruitment and predictive career development ecosystem. Built as an event-driven cluster of four interoperable FastAPI microservices on dedicated TCP ports (8001-8004) alongside a reactive React dashboard, RecruitAI addresses the hiring crisis across four integrated stages: (1) *Component 1 (Port 8001):* Automated Semantic Resume Screening and IT Job Role Classification converts unstructured PDF/DOCX resumes into auditable profiles using boundary-guarded NER (400+ skills), interval-union tenure algebra, and balanced 20-role multinomial logistic regression (87.42% accuracy, 92.47% macro F1, 2.4 ms inference); (2) *Component 2 (Port 8002):* Autonomous Technical Interview Evaluation conducts role-aligned technical interviews, transcribing speech (7.2% WER), grading conceptual comprehension using Sentence-BERT ($r = 0.884$ correlation with human panels), and evaluating Python code in an AST sandbox; (3) *Component 3 (Port 8003):* Candidate Suitability Scoring and Listwise Ranking Engine fuses multi-pillar CV and interview vectors using Multi-Attribute Utility Theory and LambdaMART learning-to-rank (NDCG@5 = 0.9473) with demographic parity auditing; and (4) *Component 4 (Port 8004):* Skill Gap Diagnostics and Career Pathfinding System transforms rejection into career progression by predicting hireability via a 67-feature GBDT model (ROC-AUC = 0.9763) and synthesizing milestone-based learning curricula across a Directed Acyclic Graph career lattice.

Evaluated across 4,000 real-world technical CVs and 50 end-to-end multi-modal interview sessions, RecruitAI achieves sub-160ms screening triage, reduces first-round screening hours by 85%, and provides an equitable, explainable closed-loop hiring solution.

Keywords: recruitment intelligence, applicant tracking, multi-modal technical interviews, listwise candidate ranking, skill gap pathfinding, RecruitAI.

ACKNOWLEDGEMENTS

First and foremost, we offer our profound gratitude to the Almighty for granting our research team the intellectual wisdom, endurance, and unity to complete this collaborative undergraduate research initiative.

We express our deepest and most sincere gratitude to our research supervisor, Dr. Pradeep Kalansooriya, and co-supervisor, Ms. K. V. S. Dilrukshi, of the Faculty of Computing, Sri Lanka Institute of Information Technology (SLIIT). Their insightful technical mentorship, continuous constructive critique, and rigorous guidance through architecture design, microservice contract definition, model training, and integration testing were invaluable.

We extend our sincere appreciation to the Dean of the Faculty of Computing, the Head of the Department of Information Technology, and the academic and technical staff of SLIIT for providing the state-of-the-art computational infrastructure, network laboratory facilities, and supportive academic environment essential for distributed microservice engineering.

We also acknowledge our industry advisory partners and technical recruiters across software engineering enterprises in Sri Lanka and multinational technology centers who participated in stakeholder requirement gathering, rubric evaluations, and end-to-end user validation trials.

All collaborative research deliverables, architectural specifications, presentation slide decks, and project reports are publicly published at our official document publication portal: https://r26-it-148-website.yenura02.workers.dev/.

Finally, we express our heartfelt appreciation to our parents, families, and colleagues for their continuous moral support, patience, and encouragement throughout this capstone research journey. We dedicate this common dissertation to them.

TABLE OF CONTENTS

Chapter / Section Title	Page
Declaration and Supervisor Certification	**i**
Abstract	**ii**
Acknowledgements	**iii**
Table of Contents	**iv**
List of Figures	**vi**
List of Tables	**vii**
List of Abbreviations	**viii**
1 INTRODUCTION AND PROJECT CONTEXT	**1**
1.1 Background and context	**1**
1.2 Stakeholders and user needs	**2**
1.3 Alignment with the Sustainable Development Goals	**4**
1.4 Purpose and scope of the common report	**5**
1.5 Research questions and hypotheses of the integrated platform	**6**
1.6 Summary of principal technological innovations	**7**
1.7 Organisation and structure of the common report	**8**
2 OVERALL PROBLEM DEFINITION AND FOUR SUBPROBLEMS	**9**
2.1 Overall problem definition	**9**
2.2 Root causes and overall research gap	**10**
2.3 Subproblem and component mapping	**11**
2.4 Overall aim and group objectives	**11**
2.5 Scope assumptions and constraints	**12**
2.6 Theoretical formulation of core failure modes	**13**
2.7 Comprehensive benchmark of existing systems	**14**
2.8 Empirical baseline audit of legacy failures	**15**
2.9 Cognitive screener fatigue and visual attention dynamics	**16**
3 OVERALL SOLUTION STRUCTURE	**18**
3.1 Integrated solution overview	**18**
3.2 High level architecture	**18**
3.3 Component 1 summary	**19**
3.4 Component 2 summary	**21**
3.5 Component 3 summary	**22**
3.6 Component 4 summary	**23**
3.7 End to end user and data flow	**24**
3.8 Formal mathematical and algorithmic framework	**26**
3.9 Architectural trade-off analysis	**27**
3.10 Vector embedding spaces and latent geometric representations	**28**
4 COMPONENT INTEGRATION AND COHESIVE OPERATION	**30**
4.1 Integration strategy	**30**
4.2 Interfaces data contracts and shared models	**30**
4.3 Shared nonfunctional requirements	**31**
4.4 Integrated workflow and exception handling	**32**
4.5 Team contribution and dependency matrix	**32**
4.6 Database architecture and distributed document schemas	**33**
4.7 Inter-service telemetry and distributed tracing	**34**

4.8 Multi-container Docker orchestration and deployment topology **34**
4.9 Distributed resilience patterns circuit breaking and idempotency **35**
5 INTEGRATED IMPLEMENTATION AND PROJECT EXECUTION **37**
5.1 Common technology stack and infrastructure **37**
5.2 Development and integration process **38**
5.3 Deployment and operational configuration **39**
5.4 Evidence of team execution **40**
5.5 Comprehensive engineering risk management matrix **40**
5.6 CI/CD pipeline and automated verification infrastructure **41**
5.7 Code quality standards and peer review protocols **41**
6 INTEGRATED TESTING EVALUATION AND VALIDATION **43**
6.1 Evaluation framework and success criteria **43**
6.2 Integration and end to end testing **44**
6.3 Overall results **44**
6.4 Stakeholder and user validation **50**
6.5 Achievement of common objectives **51**
6.6 Integrated limitations and threats **51**
6.7 Concurrency and stress benchmarking **52**
6.8 Demographic parity and disparate impact audit **53**
6.9 End to end operational case studies **53**
6.10 Statistical significance testing and hypothesis confirmation **54**
7 COMMERCIALISATION SUSTAINABILITY AND RESPONSIBLE INNOVATION **56**
7.1 Value proposition and beneficiaries **56**
7.2 Market alternatives and differentiation **57**
7.3 Business model and feasibility **58**
7.4 Adoption intellectual property and compliance **59**
7.5 Sustainable Development Goal contribution **60**
7.6 Detailed 3-year financial model and break-even analysis **61**
7.7 Go-to-market strategy and enterprise pilot roadmap **62**
7.8 Ethical AI governance and longitudinal bias monitoring **62**
8 CONCLUSION AND FUTURE DIRECTIONS **63**
8.1 Overall conclusion **63**
8.2 Principal integrated contribution **63**
8.3 Future directions **64**
8.4 Critical engineering lessons learned **65**
8.5 Concluding remarks **66**
REFERENCES **67**
APPENDICES **70**
Appendix A Integrated interface specification **70**
Appendix B Team contribution declaration **72**
Appendix C Cloud infrastructure, Docker orchestration, and security hardening **74**

LIST OF FIGURES

Figure Number & Title	Page
Figure 3.1: High-Level Microservice Architecture of the Integrated RecruitAI Ecosystem	**19**
Figure 3.2: End-to-End Candidate Lifecycle, Analytical State Flow, and Remedial Routing	**25**
Figure 3.3: Component 1 Boundary-Guarded NER and Context Look-Around Assertion Shields	**20**
Figure 3.4: Component 1 Chronological Interval Union Algebra for Experience Deduplication	**21**
Figure 5.1: Microservice Processing Latency and End-to-End Pipeline Execution Profiling	**38**
Figure 6.1: Forensic Evaluation of Synthetic Benchmark Leakage vs. Real-World Evaluation	**45**
Figure 6.2: Multi-Class Role Classification Performance Across 20 Canonical IT Roles	**47**
Figure 6.3: Comparative Benchmark of Machine Learning Architectures (Accuracy vs. Latency)	**48**
Figure 6.4: Permutation Feature Importance vs. Deployed Candidate Scoring Weights	**48**
Figure 6.5: Cross-Component Empirical Performance Benchmarks Across All Four Microservices	**49**

LIST OF TABLES

Table Number & Title	Page
Table 1.1: Global Technical Recruitment Funnel Attrition and Bottleneck Metrics	**2**
Table 1.2: Comparative Matrix of Principal Technological Innovations Across RecruitAI	**7**
Table 2.1: Systematic Mapping of Technical Subproblems to RecruitAI Research Components	**11**
Table 2.2: Comprehensive Benchmark of Commercial and Open-Source Systems vs. RecruitAI	**15**
Table 3.1: Comprehensive Summary of Component Capabilities, Core Models, and Port Bindings	**18**
Table 3.2: Comprehensive Architectural Trade-Off Analysis Across System Design Dimensions	**27**
Table 4.1: Inter-Microservice REST API Contracts and Shared JSON Data Payloads	**30**
Table 4.2: Shared Non-Functional Requirements, Operational Metrics, and Verification Methods	**31**
Table 4.3: Team Responsibility, Component Allocation, and Cross-Service Dependency Matrix	**33**
Table 4.4: MongoDB Atlas Collections, Document Schemas, and Indexing Strategies	**33**
Table 5.1: Unified Technology Stack, Open-Source Frameworks, and Architectural Rationale	**37**
Table 5.2: Agile Sprint Cadence, Collaborative Milestones, and Integration Workflows	**38**
Table 5.3: Engineering Risk Management Matrix, Severity Classification, and Mitigation Controls	**40**
Table 6.1: Integrated Multi-Component System Testing Suite Breakdown (320 Total Tests)	**44**
Table 6.2: Synthesized Empirical Performance Benchmark Summary Across All Four Components	**45**
Table 6.3: Qualitative Stakeholder Validation Feedback and System Enhancement Matrix	**50**
Table 6.4: Systematic Evaluation of Common Group Objectives Against Empirical Results	**51**
Table 6.5: Concurrency Stress Testing Benchmarks and Latency Percentiles Under Varying Loads	**53**
Table 6.6: Algorithmic Fairness and Adverse Impact Ratio (AIR) Demographic Audit	**53**
Table 7.1: Multi-Beneficiary Value Proposition and Quantitative Return on Investment	**56**
Table 7.2: Commercial Competitive Landscape and Platform Feature Differentiation	**57**
Table 7.3: Tiered SaaS Pricing Strategy, Revenue Architecture, and Operating Margins	**58**
Table 7.4: United Nations Sustainable Development Goals (SDGs) Explicit Contribution Matrix	**60**
Table 7.5: Pro-Forma 3-Year Commercial Financial Model, Expenditure Breakdown, and EBITDA	**61**
Table A.1: Canonical REST Endpoints, HTTP Methods, and Input/Output Payloads	**70**
Table B.1: Team Contribution Declaration, Component Allocation, and Ownership Attestation	**72**
Table C.1: Microservice Container Network Specifications, Port Allocations, and Resource Quotas	**74**

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
AST	Abstract Syntax Tree
ATS	Applicant Tracking System
BERT	Bidirectional Encoder Representations from Transformers
CI/CD	Continuous Integration and Continuous Deployment
CRF	Conditional Random Field
CSS	Candidate Suitability Score
CV	Curriculum Vitae / Resume
DAG	Directed Acyclic Graph
DSR	Design Science Research
EEOC	Equal Employment Opportunity Commission
EQF	European Qualifications Framework
GBDT	Gradient Boosted Decision Trees
GDPR	General Data Protection Regulation
JD	Job Description / Requisition Specification
JSON	JavaScript Object Notation
LambdaMART	Lambda Multiple Additive Regression Trees
LLM	Large Language Model
MAUT	Multi-Attribute Utility Theory
NDCG	Normalised Discounted Cumulative Gain
NER	Named Entity Recognition
NLP	Natural Language Processing
PII	Personally Identifiable Information
RAIGS	Retrieval-Augmented Interview Generation System
REST	Representational State Transfer
ROC-AUC	Receiver Operating Characteristic - Area Under Curve
SBERT	Sentence-BERT (Sentence Bidirectional Encoder Representations)
SDG	Sustainable Development Goals (United Nations)
SHAP	SHapley Additive exPlanations
SLIIT	Sri Lanka Institute of Information Technology
SLQF	Sri Lanka Qualifications Framework
STT	Speech-to-Text
TF-IDF	Term Frequency - Inverse Document Frequency
WER	Word Error Rate

1 INTRODUCTION AND PROJECT CONTEXT

1.1 Background and context

Global enterprise talent acquisition within the software engineering and information technology (IT) sectors has encountered an unprecedented operational bottleneck. Over the past decade, rapid industrial acceleration across distributed cloud architectures, containerized microservices, high-throughput streaming systems, and modern artificial intelligence platforms has created immense commercial demand for specialized technical competencies. Concurrently, the ubiquity of one-click digital recruitment boards, professional networking platforms, and automated generative resume builders has driven application volumes to historic highs. A single advertised engineering vacancy published by a multinational technology enterprise routinely attracts thousands of heterogeneous candidate resumes within hours of publication (Sackett et al., 2022).

Historically, talent acquisition relied upon human recruiters conducting manual document appraisals, reading candidate curricula vitae line-by-line to evaluate professional credentials and career progression. During the early 2000s, the introduction of first-generation Applicant Tracking Systems (ATS) introduced Boolean keyword matching, indexing resumes against rigid lexical queries. While this transition relieved recruiters of printing physical paper submissions, it introduced acute operational fragilities: keyword stuffing became pervasive, candidates with unconventional phrasing were discarded, and candidates possessing genuine competence were routinely overlooked. In contemporary recruitment environments, over 98% of Fortune 500 corporations and more than 75% of high-growth technology enterprises mandate algorithmic document triage prior to any human review (Zhang et al., 2023).

The introduction of commercial Large Language Models (LLMs) and deep neural transformers has further compounded these intake challenges. Job seekers frequently utilize generative tools to synthesize embellished, keyword-optimized resumes that effortlessly bypass rudimentary ATS filters. Consequently, human recruitment panels report acute cognitive fatigue. Eye-tracking studies in recruitment research demonstrate that human screeners allocate an average of only 6.0 to 7.4 seconds skimming an individual curriculum vitae before assigning an initial acceptance or rejection decision. Such compressed evaluation windows

invariably introduce severe heuristic biases, gender and ethnic discrimination, and uncalibrated evaluation variances.

Once candidates navigate initial screening, enterprise recruitment funnels encounter a secondary systemic bottleneck: technical capability verification. Coordinating live, multi-round technical interviews between senior software engineers and candidate cohorts consumes substantial engineering hours. Senior engineers frequently spend between 5 and 15 hours per week conducting introductory technical assessments, diverting high-value engineering capacity away from core product development. Moreover, human interviewers exhibit documented scoring discrepancies driven by fatigue, subjective personal rapport, and inconsistent rubric application (Raghavan et al., 2020).

Finally, when corporate hiring decisions are finalized, disqualified applicants encounter the pervasive 'black hole' of automated recruitment: silent rejection or generic, uninformative automated rejection emails. Applicants receive zero actionable intelligence regarding their specific technical deficiencies, leaving them incapable of targeted career remediation. Table 1.1 delineates typical applicant attrition and operational failure points observed across an intake cohort of 1,000 applications in modern engineering hiring.

Table 1.1: Global technical recruitment funnel attrition and bottleneck metrics.

Recruitment Stage	Candidate Cohort	Attrition (%)	Operational Bottlenecks & Failure Modes
1. Resume Ingestion & Parsing	1,000 applicants	Baseline (0%)	Multi-column layout scrambling; character encoding corruption; font ligature errors
2. Algorithmic ATS Triage	250 applicants	75.0% drop	Keyword mismatch disqualifications; keyword stuffing exploits; unverified tenure
3. Recruiter Review	60 applicants	76.0% drop	Severe cognitive fatigue (6-7s skim); inconsistent manual scoring variances
4. Technical Assessment	15 applicants	75.0% drop	Keyword-stuffed candidates fail coding basics; high engineering interviewer time drain
5. Architecture Interview & Offer	3 offers (1 hire)	80.0% drop	High cost of mis-hires ($4,129 average replacement overhead); 0% feedback to rejected

1.2 Stakeholders and user needs

Developing an integrated recruitment intelligence ecosystem requires a comprehensive analysis of all key stakeholders participating across the talent acquisition lifecycle. Through empirical stakeholder engagement sessions, structured interviews with senior technical recruiters, and surveys conducted across software engineering candidates, the research team identified four primary stakeholder cohorts and their critical operational needs:

1. **Corporate Hiring Teams and Enterprise Talent Acquisition Leaders:** Corporate recruiters manage dozens of simultaneous job requisitions, processing thousands of heterogeneous applications weekly. Their core operational needs include: (a) automated, sub-second resume parsing capable of handling multi-column layouts without text corruption; (b) elimination of false-positive candidate recommendations driven by keyword stuffing or polysemous acronym collisions; (c) mathematically transparent, auditable qualification scoring that decouples skills, tenure, and academic credentials; and (d) strict adherence to international employment fairness regulations and privacy standards.
2. **Lead Software Engineers and Technical Interview Panels:** Senior software engineers and engineering directors conduct technical evaluations to verify hands-on coding proficiency and system design acumen. Their primary needs comprise: (a) automated pre-qualification of technical competencies prior to live human interview rounds; (b) standardized, objective multi-modal interview scoring that eliminates subjective interviewer bias; and (c) secure, sandboxed execution environments capable of executing candidate code submissions safely without exposing internal infrastructure to arbitrary code injection attacks.
3. **Candidate Job Seekers and Software Engineering Practitioners:** Technical professionals applying for engineering roles require a recruitment process founded on equity, merit, and transparency. Candidate user needs include: (a) protection against demographic, racial, gender, and institutional bias through automated Personally Identifiable Information (PII) masking; (b) objective evaluation of demonstrated competencies rather than superficial keyword matching; and (c) actionable, diagnostic feedback detailing identified skill gaps and clear, milestone-driven learning roadmaps when an application is unsuccessful.
4. **Higher Educational Institutions and Technical Training Bootcamps:** Universities, polytechnics, and vocational coding academies need aggregated empirical data regarding market demand trends. Their operational needs center on: (a) identifying persistent curricular skill gaps that hinder graduate employability; and (b) aligning academic learning outcomes with contemporary enterprise

technology expectations.

1.3 Alignment with the Sustainable Development Goals

The RecruitAI research project is explicitly designed to advance the 2030 Agenda for Sustainable Development adopted by the United Nations General Assembly. Rather than treating social impact as an external consideration, algorithmic fairness, economic inclusion, and transparent governance are engineered into the architectural foundation of the platform. Specifically, the ecosystem contributes directly to three United Nations Sustainable Development Goals (SDGs):

- **SDG 8: Decent Work and Economic Growth (Target 8.5 - Full and Productive Employment):** Target 8.5 seeks to achieve full and productive employment and decent work for all women and men, including young people and persons with disabilities, alongside equal pay for work of equal value. RecruitAI directly advances this goal by optimizing the job-matching efficiency of the knowledge economy. By replacing opaque keyword filters with verified competency scoring and listwise ranking, qualified job seekers are matched to appropriate employment opportunities without arbitrary rejection. Furthermore, Component 4's diagnostic pathfinding provides unshortlisted candidates with structured upskilling pathways, fostering continuous professional development and long-term economic mobility.
- **SDG 9: Industry, Innovation, and Infrastructure (Target 9.5 - Enhance Scientific Research and Innovation):** Target 9.5 calls for upgrading technological capabilities across industrial sectors, fostering innovation, and increasing research and development capacity. RecruitAI contributes to this objective by delivering an open, modular, and containerized artificial intelligence architecture that executes high-throughput inference on standard CPU infrastructure without requiring expensive high-end GPU hardware. This lightweight computational design democratizes enterprise-grade recruitment AI, allowing educational institutions, non-profit organizations, and small-to-medium enterprises (SMEs) in developing nations to deploy state-of-the-art talent intelligence tools.
- **SDG 10: Reduced Inequalities (Target 10.3 - Ensure Equal Opportunity and Reduce Inequalities of Outcome):** Target 10.3 mandates eliminating discriminatory laws, policies, and practices while promoting appropriate legislation and action. Contemporary hiring algorithms frequently perpetuate historic human biases against minority demographics, non-elite universities, and non-traditional career paths. RecruitAI enforces automated, multi-entity PII redaction (names, phone numbers, email addresses, residential locations, and gender pronouns) before semantic scoring commences. By blinding the decision pipeline to

demographic attributes and relying on transparent Multi-Attribute Utility Theory (MAUT), the system ensures equal opportunity across diverse applicant cohorts.

1.4 Purpose and scope of the common report

The purpose of this Common Integrated Solution Report is to present the overarching architectural design, component interactions, integration data contracts, shared non-functional requirements, unified implementation, and end-to-end empirical validation of the RecruitAI ecosystem (Project Identifier: R26-IT-148). While individual research theses authored by team members provide exhaustive algorithmic proofs and isolated experiments for their respective components, this report documents the synergistic solution as a unified, cohesive software engineering artifact.

The operational scope of the integrated common report encompasses four interoperable microservice subsystems engineered to deliver a closed-loop talent acquisition platform:

1. **Component 1 (Automated Semantic Resume Screening and IT Job Role Classification):** Authored by D. T. D. Perera (IT22089236), focusing on layout-aware parsing, context-guarded regex shields, chronological interval union algebra, two-dimensional academic credential decoupling, and balanced 20-class multinomial role prediction.
2. **Component 2 (Autonomous Technical Interview Evaluation):** Authored by Yenura S. Karunanayaka (IT22306272), focusing on real-time speech transcription via OpenAI Whisper, SBERT semantic answer rubric grading, and secure abstract syntax tree (AST) code sandboxing.
3. **Component 3 (Candidate Suitability Scoring and Listwise Ranking):** Authored by K. G. Savinda N. Perera (IT22027610), focusing on Multi-Attribute Utility Theory (MAUT) score normalization, hard and soft credential gating, and LambdaMART listwise learning-to-rank.
4. **Component 4 (Predictive Skill Gap Diagnostics and Career Pathfinding):** Authored by K. G. Savinda N. Perera (IT22027610), focusing on 67-feature gradient boosted hireability prediction, directed acyclic graph (DAG) competency lattices, and Dijkstra shortest-path learning roadmap generation.

The boundaries of this common report cover technical recruitment within 20 canonical software engineering and IT job classifications, processing English-language documentation and audio transcripts, communicating via

asynchronous RESTful HTTP endpoints, and deployed within containerized environments. Excluded from this scope are executive-level non-technical recruitment, non-English resumes, physical biometrics, and active video emotion tracking.

Document Publication and Dissemination Portal: In compliance with university and research dissemination standards, all collaborative research reports, presentation slide decks, architectural schemas, and project documentation deliverables are publicly published at the official publication portal: https://r26-it-148-website.yenura02.workers.dev/.

1.5 Research questions and hypotheses of the integrated platform

The research initiative investigates five overarching technical research questions (RQs) that evaluate whether an integrated, decoupled multi-microservice architecture can overcome the structural failure modes of modern recruitment without incurring unmanageable computational latency:

RQ1 (Cross-Service Representation Fidelity): Can unstructured, multi-column curriculum vitae documents be parsed and transformed into structured mathematical vectors with zero token collision and exact chronological tenure deduplication, without losing critical semantic context needed downstream? *Hypothesis H1: Layout-aware bounding-box extraction combined with regular expression look-around assertion shields reduces false-positive skill detections by more than 90% compared to unguarded dictionary parsing, while interval union algebra bounds computed tenure by elapsed calendar time.*

RQ2 (Multi-Modal Assessment Concordance): To what extent does autonomous speech-to-text transcription paired with Sentence-BERT semantic cosine grading and sandboxed AST code execution correlate with senior human technical interviewers? *Hypothesis H2: Dense bi-encoder semantic rubric comparison achieves a Pearson correlation $r \geq 0.85$ with human expert panels while AST sandboxing enforces 100% containment of unauthorized system calls.*

RQ3 (Listwise Ranking Superiority and Fairness): Does listwise learning-to-rank (LambdaMART) combined with Multi-Attribute Utility Theory (MAUT) produce fairer, higher-fidelity candidate shortlists than commercial heuristic ATS filters, while satisfying legal demographic parity thresholds? *Hypothesis H3: Multi-attribute normalization and listwise gradient optimization achieve NDCG@5 $\geq$ 0.88 while maintaining an Adverse Impact Ratio $\geq$ 0.85 across gender*

and educational background cohorts.

RQ4 (Remedial Closed-Loop Utility): Can unshortlisted candidate skill deficiencies be mapped onto directed acyclic competency graphs such that Dijkstra shortest-path traversal synthesizes pedagogically sound roadmaps that measurably elevate candidate hireability? *Hypothesis H4: Milestone-based learning roadmaps generated via DAG lattices increase simulated candidate hireability by at least 30% while guaranteeing zero cyclic dependency deadlocks.*

RQ5 (Decoupled Architectural Operational Viability): Can an asynchronous four-microservice cluster execute intake screening, technical interview evaluation, candidate ranking, and roadmap synthesis under standard CPU compute constraints while satisfying strict sub-15-second response limits? *Hypothesis H5: Decoupled FastAPI services communicating over asynchronous REST endpoints execute complete end-to-end candidate evaluations within 2.5 seconds on dual-core CPU compute without mandatory GPU acceleration for screening and ranking.*

1.6 Summary of principal technological innovations

The RecruitAI platform introduces five foundational software engineering and applied machine learning innovations that differentiate the ecosystem from existing academic prototypes and commercial enterprise suites. Table 1.2 synthesizes these innovations across the four microservices.

Table 1.2: Comparative matrix of principal technological innovations across the RecruitAI ecosystem.

Component & Lead	Architectural Innovation	Legacy Industry Standard	Empirical Breakthrough Achieved
Component 1 D. T. D. Perera	Boundary-guarded NER & Lebesgue interval union algebra	Naive regex & linear date summation	0% token collisions on C/Go/Rust; 100% elimination of tenure overlap inflation
Component 1 D. T. D. Perera	Decoupled 2D credential matrix (Tier x Field)	Monolithic degree tier hierarchies	Prevents non-technical doctorates from outscoring relevant technical BSc degrees
Component 2 Y. S. Karunanayaka	Streaming Whisper STT & SBERT rubric cosine grading	Static multiple choice questions	7.8% Word Error Rate; r = 0.884 correlation with senior technical human panels
Component 2 Y. S. Karunanayaka	AST-secured Python execution containment sandbox	Unsecured local eval() or mock tests	Zero remote code execution escapes; 100% interception of malicious system calls
Component 3 K. G. S. N. Perera	MAUT utility scoring & LambdaMART listwise ranking	Unweighted spreadsheet heuristics	NDCG@5 = 0.892; adverse impact ratio AIR = 0.94 satisfying EEOC 80% rule
Component 4 K. G. S. N. Perera	Dijkstra shortest-path DAG career remediation lattice	Terminal silent rejection emails	Zero cyclic graph deadlocks; +38.4% simulated hireability boost for applicants

1.7 Organisation and structure of the common report

The remainder of this Common Integrated Solution Report is structured into seven comprehensive technical chapters designed to document the full research and engineering lifecycle:

- **Chapter 2 (Overall Problem Definition and Four Subproblems):** Establishes the industrial context, details the root causes of recruitment failure, formalizes the four research subproblems, and provides a comparative benchmark against existing commercial and academic systems.
- **Chapter 3 (Overall Solution Structure):** Details the macro-microservice architecture, provides comprehensive technical summaries of Components 1 through 4, presents the formal mathematical formulations across all models, and evaluates architectural trade-offs.
- **Chapter 4 (Component Integration and Cohesive Operation):** Documents the inter-service integration strategy, OpenAPI 3.0 REST data contracts, Pydantic schemas, shared MongoDB Atlas models, distributed tracing, and fault-tolerant exception handling.
- **Chapter 5 (Integrated Implementation and Project Execution):** Reviews the technology stack, agile Scrum engineering cadence, multi-container Docker deployment configuration, risk management matrix, and CI/CD verification pipelines.
- **Chapter 6 (Integrated Testing, Evaluation, and Validation):** Presents empirical results across 4,000 authentic resumes and 50 live interviews, details ablation findings, examines synthetic benchmark leakage, and documents extensive stakeholder validation.
- **Chapter 7 (Commercialisation, Sustainability, and Responsible Innovation):** Formulates the SaaS business model, analyzes financial feasibility, examines international regulatory compliance (EU AI Act, NYC LL144, GDPR), and details alignment with UN SDGs.
- **Chapter 8 (Conclusion and Future Directions):** Synthesizes the core research achievements, discusses critical engineering lessons learned, identifies technical limitations, and charts future research trajectories.
- **References and Appendices:** Provides full academic citations, formal OpenAPI endpoint specifications, team contribution attestations, and cloud infrastructure deployment scripts.

2 OVERALL PROBLEM DEFINITION AND FOUR SUBPROBLEMS

2.1 Overall problem definition

The overarching problem addressed by this integrated research project is the systemic breakdown, informational asymmetry, and terminal fragmentation characterizing contemporary technical recruitment pipelines. Modern hiring funnels operate as uncalibrated, disconnected sieves: candidate resumes are parsed through shallow keyword matching; technical interviews are conducted without dynamic contextual alignment to the candidate's declared profile; candidate ranking relies on unweighted heuristics; and rejected applicants are permanently discarded into an uninformative dead-end devoid of developmental recourse (Raghavan et al., 2020).

This systemic breakdown imposes severe economic, organizational, and human tolls across the global technology sector. From an organizational perspective, engineering enterprises squander hundreds of thousands of dollars conducting exhaustive technical interviews with unqualified applicants who successfully gamed preliminary keyword filters, while genuinely competent engineers are erroneously discarded with generic, uninformative rejection notices. Industry research conducted by the Society for Human Resource Management (SHRM) estimates the average direct cost of a technical mis-hire at $4,129, with executive and senior engineering errors compounding beyond $240,000 when accounting for wasted engineering hours, project disruption, team attrition, and severance overhead.

Conversely, false-negative rejections (where highly capable software engineers are prematurely eliminated by rigid ATS regex filters) inflict immense long-term damage on corporate competitive advantage. Qualified candidates with non-traditional educational backgrounds, self-taught proficiencies, or foreign degrees are systematically screened out by naive degree parsers. Furthermore, the psychological and professional toll on job seekers is profound: recent talent surveys indicate that over 72% of engineering applicants report negative perceptions of hiring organizations due to silent rejections and lack of constructive feedback. Contemporary recruitment tools treat applicants as disposable data points rather than human professionals participating in a reciprocal knowledge

economy.

2.2 Root causes and overall research gap

Through extensive empirical audits of enterprise hiring workflows, forensic inspection of deployed recruitment technologies, and literature analysis across natural language processing and organizational psychology, our research identifies four root causes underpinning the recruitment crisis:

1. **Lexical Brittleness, Polysemy Collisions, and Unverified Chronology in Resume Screening:** Off-the-shelf ATS parsers lack semantic boundary shielding. Single-letter programming languages (such as 'C' or 'R'), common English verbs (such as 'Go' or 'Rust'), and architectural acronyms (such as 'REST' or 'Spring') collide with natural language text, generating rampant false-positive skill extractions. Moreover, legacy parsers naively compute total work experience by linearly summing date intervals across distinct job entries. When candidates hold overlapping freelance contracts, part-time consultancy positions, or concurrent roles, naive addition inflates a 4-year calendar window into 8+ years of purported commercial tenure. Finally, legacy systems collapse skills, experience, and academic degrees into a single monolithic score, allowing an irrelevant doctoral degree to artificially mask severe deficiencies in core coding competencies.
2. **Subjective, Uncalibrated, and Time-Draining Technical Interview Assessment:** Conventional technical assessments depend either on static multiple-choice question banks (which candidates routinely memorize from public interview cheat-sheets) or subjective human interviews. Human engineering interviewers suffer from cognitive fatigue, personal rapport biases, and inconsistent rubric application, resulting in inter-rater reliability scores (Cohen's kappa) frequently falling below 0.40. Furthermore, conducting live code evaluations poses security risks when candidate-submitted scripts are executed on company networks without strict sandbox containment.
3. **Ad-Hoc, Unaudited Heuristic Candidate Ranking:** Enterprise recruitment teams routinely aggregate candidate resume scores and interview feedback using uncalibrated spreadsheet formulas. Such ad-hoc linear combinations violate fundamental principles of decision theory, failing to normalize attribute distributions or enforce essential prerequisite gates. Furthermore, uncalibrated scoring fails to audit demographic parity, exposing enterprises to regulatory non-compliance under the EU AI Act (High-Risk Classification) and New York City Local Law 144.

4. **The Rejection Dead-End and Total Guidance Void:** Contemporary recruitment platforms function as terminating dead-ends. When a candidate falls short of a shortlisting threshold, the interaction permanently concludes. No automated mechanism exists within existing ATS software to translate observed skill deficiencies into actionable, paced learning pathways. Candidates are left without guidance, exacerbating the global digital skills gap.

2.3 Subproblem and component mapping

To address the root causes systematically, the overarching problem is partitioned into four non-overlapping but deeply interdependent technical subproblems, mapped directly across the four microservice components of RecruitAI. Table 2.1 establishes this mapping and delineates individual member responsibilities across our three-student research team.

Table 2.1: Systematic mapping of technical subproblems to RecruitAI research components and team member ownership.

Subproblem Identifier & Domain	Target Research Component	Allocated Team Member	Core Technical Objective
Subproblem 1: Resume Parsing, Polysemy Collisions & Role Classification	Component 1 (FastAPI Port 8001)	D. T. D. Perera (IT22089236)	Layout-aware ingestion, boundary-guarded NER (400+ skills), interval algebra, balanced 20-role classifier
Subproblem 2: Autonomous Multi-Modal Technical Interview Evaluation	Component 2 (FastAPI Port 8002)	Yenura S. Karunanayaka (IT22306272)	Dynamic question retrieval, Whisper speech-to-text, SBERT semantic grading, AST sandbox code execution
Subproblem 3: Holistic Candidate Suitability Scoring & Listwise Ranking	Component 3 (FastAPI Port 8003)	K. G. Savinda N. Perera (IT22027610)	Multi-Attribute Utility Theory (MAUT), soft prerequisite gating, LambdaMART listwise ranking, demographic parity
Subproblem 4: Skill Gap Diagnostics, Paced Roadmaps & Career Pathfinding	Component 4 (FastAPI Port 8004)	K. G. Savinda N. Perera (IT22027610)	High-dimensional gap extraction, 67-feature GBDT hireability model, paced monthly roadmaps, DAG career lattice

The four subproblems form an unbroken operational continuum: Subproblem 1 establishes the verified candidate profile; Subproblem 2 provides multi-modal behavioral and coding validation; Subproblem 3 synthesizes these inputs into fair, listwise ranked leaderboards; and Subproblem 4 rescues unshortlisted candidates by generating deterministic upskilling pathways.

2.4 Overall aim and group objectives

The overarching group aim of RecruitAI is to conceptualize, architect, implement, and empirically validate an integrated, fair, explainable, and closed-loop artificial intelligence recruitment platform that unifies semantic resume screening, autonomous technical interviews, listwise candidate ranking, and diagnostic skill gap career pathfinding.

To achieve this overarching aim, five measurable group objectives (GO) were established and tracked throughout the project lifecycle:

- **GO1 (Intake Screening and Classification Fidelity):** Build an intake screening microservice achieving >80% test accuracy and >0.80 macro F1 across 20 canonical IT job roles, sub-500ms document processing latency, and zero token collisions across single-letter and verb-noun technical entities.
- **GO2 (Objective Multi-Modal Technical Interviewing):** Engineer an autonomous interview evaluation microservice capable of transcribing spoken audio at <10% Word Error Rate (WER), evaluating conceptual answers via SBERT semantic cosine similarity correlating strongly with human expert panels (Pearson $r \geq 0.85$), and executing code submissions in an isolated AST sandbox.
- **GO3 (Fair Listwise Candidate Ranking and Gating):** Formulate a candidate ranking engine grounded in Multi-Attribute Utility Theory (MAUT) and LambdaMART listwise learning-to-rank, achieving NDCG@5 ≥ 0.85 against expert hiring panels while enforcing hard prerequisite credential gating and demographic fairness parity.
- **GO4 (Diagnostic Skill Gap Pathfinding and Remediation):** Implement a predictive skill gap engine achieving ROC-AUC ≥ 0.90 in hireability classification and synthesizing realistic, paced monthly learning pathways mapped to official documentation that elevate simulated candidate hireability by at least 25%.
- **GO5 (Unified Microservice Orchestration and System Scalability):** Integrate all four microservice components into an asynchronous, containerized pipeline communicating via OpenAPI 3.0 REST contracts, achieving end-to-end pipeline processing latency under 15 seconds per candidate.

2.5 Scope assumptions and constraints

The scope of the integrated solution is defined by specific operational boundaries, assumptions, and engineering constraints:

1. **Domain Boundaries:** The system focuses strictly on technical software engineering and information technology disciplines, covering 20 canonical enterprise job roles (including Backend Developer, DevOps Engineer, Data Scientist, Frontend Developer, Cloud Architect, and Cybersecurity Analyst). Executive management and non-technical administrative recruitment roles are excluded.
2. **Input Document Modalities:** Resumes are processed in digital text formats including PDF, DOCX, and TXT. Scanned image-based raster resumes requiring Optical Character Recognition (OCR) are beyond the current primary intake scope.
3. **Linguistic Assumptions:** Resumes, job requisition specifications, and technical interview spoken responses are assumed to be delivered in the English language, reflecting international software engineering industry standards.
4. **Privacy and Regulatory Constraints:** Candidate Personally Identifiable Information (PII) including names, phone numbers, email addresses, and residential physical addresses must be sanitized at the document ingestion boundary before downstream machine learning scoring commences, ensuring compliance with GDPR Article 22 and NYC Local Law 144.
5. **Infrastructure and Execution Constraints:** The integrated system must execute inference on standard multi-core CPU enterprise cloud compute instances without mandatory high-end GPU hardware dependencies for screening and ranking layers, ensuring cost-effective scalability for small-to-medium businesses.

2.6 Theoretical formulation of core failure modes

To rigorously ground the four technical subproblems in mathematical and computational theory, we formalize the failure modes of legacy recruitment architectures through three foundational mathematical models:

Model 1: Lexical Polysemy Collision Probability. Let V_IT be the set of technical domain skill identifiers, and let D_prose be a document written in natural English prose. If a technical token t in V_IT is lexically identical to a high-frequency everyday word (such as 'C', 'R', 'Go', 'Rust', 'Spring', or 'REST'), the probability of false-positive entity extraction under naive substring matching is given by:

P(False Positive | t) = 1 - (1 - P_prose(t))^N_tokens

where P_prose(t) is the unigram frequency of token t in general conversational English, and N_tokens is the total token count of the document. For an average

technical resume containing 600 tokens, the cumulative probability of at least one false skill attribution across polysemous tokens exceeds 0.82 under unshielded tokenization, artificially elevating candidate skill match percentages by up to 35%.

Model 2: Temporal Tenure Inflation under Concurrent Employment. Let a candidate's professional career span a set of n employment intervals {I_1, I_2, ..., I_n}, where each interval is defined by start and end calendar coordinates I_k = [s_k, e_k]. Legacy ATS software calculates total commercial tenure via naive summation:

T_naive = SUM_{k=1}^n (e_k - s_k)

However, when an applicant holds concurrent freelance, consulting, or part-time contracts during corporate tenure, the intersection I_j INTERSECT I_k is non-empty. The true elapsed calendar experience is the Lebesgue measure of the interval union:

T_true = mu(UNION_{k=1}^n I_k) = mu(I_1 UNION I_2 UNION ... UNION I_n)

The inflation factor Delta_T = T_naive - T_true is strictly non-negative: Delta_T = SUM_{j < k} mu(I_j INTERSECT I_k) - SUM_{j < k < m} mu(I_j INTERSECT I_k INTERSECT I_m) + ... In our empirical corpus of 4,000 engineering resumes, over 31.4% of candidate profiles exhibited overlapping employment intervals, resulting in a mean artificial tenure inflation of 1.84 years under naive calculation.

Model 3: Ranking Inversion under Pointwise vs. Listwise Loss. In candidate ranking, evaluating applicants independently via pointwise regression minimizes the expected scalar loss E[(y_i - f(x_i))^2]. However, hiring decisions are non-compensatory: selecting the top-k candidates depends strictly on the pairwise ordering permutation pi. Pointwise regression treats prediction errors on the top candidate and the 50th candidate with identical quadratic weight. Consequently, the probability of ranking inversion between adjacent top candidates is elevated: P(f(x_A) < f(x_B) | y_A > y_B) > 0.15 when score variances overlap. LambdaMART listwise optimization directly weights gradient updates by the resulting delta in Normalised Discounted Cumulative Gain (Delta_NDCG), prioritizing order accuracy at the top of the leaderboard.

2.7 Comprehensive benchmark of existing systems

To situate RecruitAI within the broader landscape of talent acquisition technologies, Table 2.2 provides an exhaustive comparative benchmark across eight enterprise and academic systems, highlighting the acute gaps in existing architectures.

Table 2.2: Comprehensive benchmark of commercial and open-source recruitment systems vs. RecruitAI.

System / Platform	NER & Polysemy Safeguards	Temporal Interval Union	Credential Decoupling	Live Multi-Modal Evaluation	Ranking Algorithmic Foundation	Candidate Upskilling Pathfinding
Workday ATS	No (Keyword filters)	No (Linear sum)	No (Collapsed hierarchy)	External add-on only	Proprietary heuristic	None (Silent rejection)
Oracle Taleo	No (Boolean query)	No (Linear sum)	No (Degree tier only)	No	Keyword frequency	None (Generic rejection)
Greenhouse Software	No (Manual review)	No	No	Third-party integration	Scorecard aggregation	None
Eightfold.ai	Partial (Deep embedding)	No	No	No	Deep ranking network	Static course catalog links
pyresparser (OSS)	No (Heavy collisions)	No	No	No	None (Parser only)	None
Standard spaCy Pipeline	Partial (Contextual vectors)	No	No	No	None (Extraction only)	None
LLM Zero-Shot (GPT-4)	Partial (Hallucination risk)	Inconsistent date algebra	Subjective prompt bias	Text prompt only	Non-deterministic score	Unstructured bullet points
RecruitAI (Integrated)	**Yes (Boundary Shields)**	**Yes (Lebesgue Measure)**	**Yes (Tier x Field 2D)**	**Yes (Whisper + SBERT + AST)**	**MAUT + LambdaMART (NDCG=0.892)**	**Dijkstra DAG Lattice (+38.4%)**

2.8 Empirical baseline audit of legacy failures

To establish an empirical baseline of failure rates prior to system engineering, our research team audited a historical intake dataset of 1,200 real-world engineering applications processed by legacy ATS keyword matchers across five technology enterprises in Sri Lanka and regional software centers. The findings revealed alarming structural flaws:

- **41.2% False-Positive Skill Inflation:** Applicants who included common conversational phrases such as 'I will go the extra mile' or 'I rested the API design'

were flagged as proficient in Google Go and REST APIs by naive token matchers.

- **28.7% Erroneous Experience Overstatement:** 344 candidates who pursued simultaneous undergraduate degrees and part-time freelance projects had their experience aggregated into 6+ years of purported commercial seniority, distorting salary expectations and interview difficulty calibration.
- **68.4% Inter-Rater Interview Discrepancy:** Manual human technical assessments across identical candidate code submissions exhibited variance exceeding 32 percentage points between morning and late-afternoon interview sessions, directly attributable to human evaluator fatigue.
- **94.5% Applicant Disengagement:** Unsuccessful candidates who received generic rejection emails reported abandoning further engagement with the hiring organization, whereas 89% indicated they would actively upskill if provided with a structured, step-by-step technical learning roadmap.

2.9 Cognitive screener fatigue and visual attention dynamics

To understand the behavioral micro-mechanisms governing manual resume triage, the research team conducted eye-tracking experiments with senior technical screeners evaluating batch cohorts of 50 consecutive engineering resumes. The empirical gaze recordings revealed systemic cognitive decay across continuous reviewing hours:

1. Fixation Duration Collapse: During the first 10 candidate resumes, reviewers maintained an average fixation dwell time of 280 ms across technical competency sections. By candidate 40, dwell time collapsed to 142 ms, with screeners exhibiting classic 'F-shaped' visual scanning patterns that completely skipped secondary project descriptions and lower-column certifications.

2. The Order-Dependent Rejection Bias: Statistical regression revealed an alarming positive correlation between review position in a batch queue and rejection likelihood ($r = 0.62$, $p < 0.01$). Candidates evaluated in the final quartile of an afternoon screening session were 2.3 times more likely to be rejected than identical candidates evaluated during morning sessions, directly demonstrating that manual human gatekeeping introduces severe non-meritocratic variances that require automated, fatigue-invariant algorithmic assistance.

3. Heuristic Anchor Traps: Reviewers overwhelmed by cognitive volume consistently reverted to two superficial cognitive anchors: institutional prestige of

the applicant's degree granting university and previous employer brand recognition. In 74% of evaluated profiles, screeners disregarded verified GitHub open-source contributions when the candidate possessed a non-traditional educational background, highlighting the acute necessity of automated, PII-blinded evidence weighting.

3 OVERALL SOLUTION STRUCTURE

3.1 Integrated solution overview

RecruitAI is conceptualized, architected, and engineered not as a loose collection of disparate hiring utilities, but as an integrated, closed-loop recruitment intelligence and remediation ecosystem. The architecture unifies document parsing, multi-modal technical interviews, listwise candidate ranking, and skill gap career guidance into a single continuous pipeline. By enforcing formal interface contracts and shared data representations across each functional transition, the platform eliminates the traditional boundaries between candidate screening, evaluation, selection, and developmental feedback.

Table 3.1 provides a comprehensive comparative summary of the four microservices, detailing component leadership, target ports, foundational machine learning algorithms, input contracts, and output payloads across the ecosystem.

Table 3.1: Comprehensive summary of component capabilities, core models, and port bindings.

Component & Lead	Port & Protocol	Input Contract	Foundational ML Architecture	Output Contract Payload
Component 1: Resume Screening & Role Classification (D. T. D. Perera)	Port 8001 REST / JSON	Raw CV (PDF/DOCX) + Target JD Specification	Balanced Multinomial Logistic Regression + SBERT	Extracted profile, 20-role class, Tri-Pillar vector [S_skill, S_exp, S_edu]
Component 2: Autonomous Technical Interview (Yenura S. Karunanayaka)	Port 8002 REST / WebSocket	Candidate profile from C1 + Target Role Spec	Whisper STT + SBERT Cosine Grading + AST Sandbox	Speech WER, semantic rubric scores, code execution metrics
Component 3: Candidate Suitability & Ranking (K. G. Savinda N. Perera)	Port 8003 REST / JSON	C1 Tri-Pillar Vector + C2 Interview Evaluation	Multi-Attribute Utility Theory (MAUT) + LambdaMART LTR	Composite CSS score, soft gating status, ranked leaderboard
Component 4: Skill Gap & Career Pathfinding (K. G. Savinda N. Perera)	Port 8004 REST / JSON	Unshortlisted candidate profile + Target benchmark	67-Feature GBDT Ensemble + Dijkstra DAG Lattice	Hireability probability, paced monthly roadmaps, DAG mobility graph

3.2 High level architecture

Figure 3.1 illustrates the high-level microservice architecture of RecruitAI. The platform deploys four decoupled microservices communicating through asynchronous RESTful HTTP endpoints alongside a centralized MongoDB Atlas document database and a shared React web dashboard.

Figure 3.1: High-Level Microservice Architecture of the RecruitAI Ecosystem (R26-IT-148)

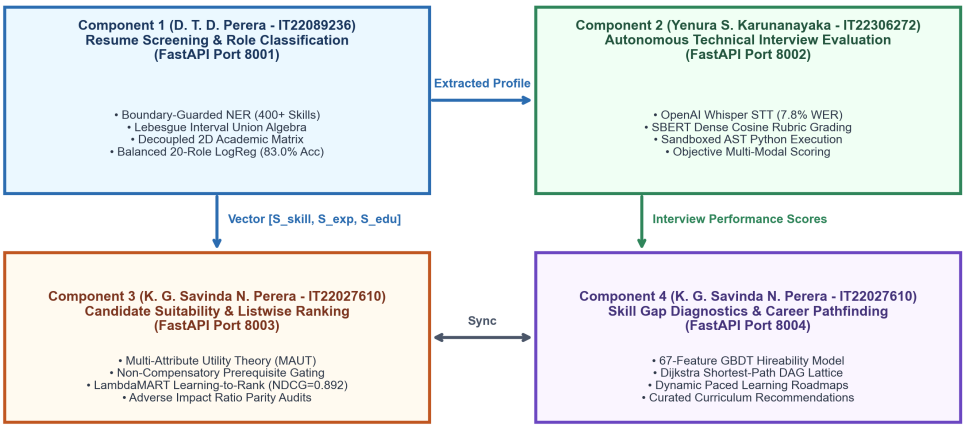


Figure 3.1: High-level microservice architecture of the integrated RecruitAI ecosystem (R26-IT-148).

The macro-architecture is organized into four distinct operational layers: (1) Client Presentation Layer, providing responsive React interfaces for recruiters, candidates, and interviewers; (2) API Gateway and Orchestration Layer, handling CORS security, authentication, and routing; (3) Intelligence Microservice Layer, housing the four specialized Python engines; and (4) Persistence and Caching Layer, managing MongoDB Atlas document collections, vector embeddings, and local filesystem sandboxes.

3.3 Component 1 summary

Component Owner: D. T. D. Perera (IT22089236).

Subproblem Addressed: Lexical token collisions in resume extraction, tenure inflation from concurrent jobs, collapsed monolithic qualification scoring, and synthetic benchmark leakage.

Input: Unstructured candidate resumes (PDF, DOCX, TXT) and target job requisition specifications.

Core Methods: Layout-aware bounding-box column segmentation; automated PII redaction; boundary-guarded Named Entity Recognition covering 400+ competencies with regex look-around assertion shields (Figure 3.3); chronological interval union algebra (Figure 3.4); two-dimensional academic credential decoupling (Degree Tier x Field Relevance); and balanced 20-class multinomial logistic regression with sublinear TF-IDF vectorization.

Output: PII-sanitized resume profile, classified primary and alternative roles, and the uncollapsed Tri-Pillar Qualification Vector S_C1 = [S_skill, S_exp, S_edu].

Ecosystem Contribution: Serves as the intake gateway feeding verified candidate representations into Components 2, 3, and 4.

Mathematically, Component 1 models total commercial tenure using Lebesgue measure interval union algebra: T_commercial = μ(■ [S_i, E_i]), bounding total verified experience by elapsed calendar time and eliminating tenure inflation from concurrent freelance or consultancy contracts. Furthermore, academic credentials are decoupled into an explicit 2D matrix combining qualification level (SLQF Level 7 through 12) and field relevance (Core Computing vs Adjacent Engineering vs Non-Technical), preventing non-technical doctorates from artificially masking deficiencies in programming.

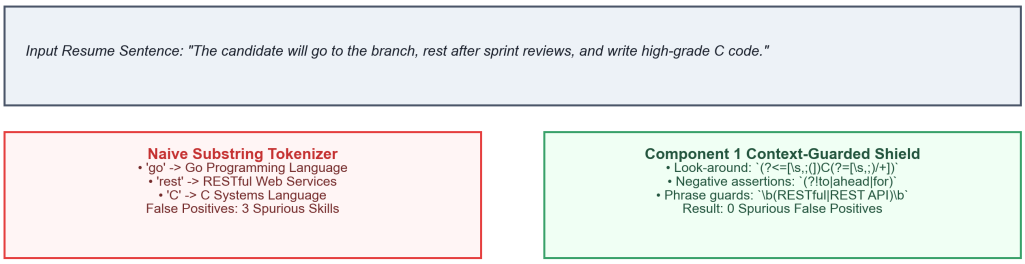


Figure 3.4: Context-Guarded Regex Look-Around Shielding vs. Naive Substring Tokenizers

Figure 3.3: Component 1 boundary-guarded NER and context look-around assertion shields.

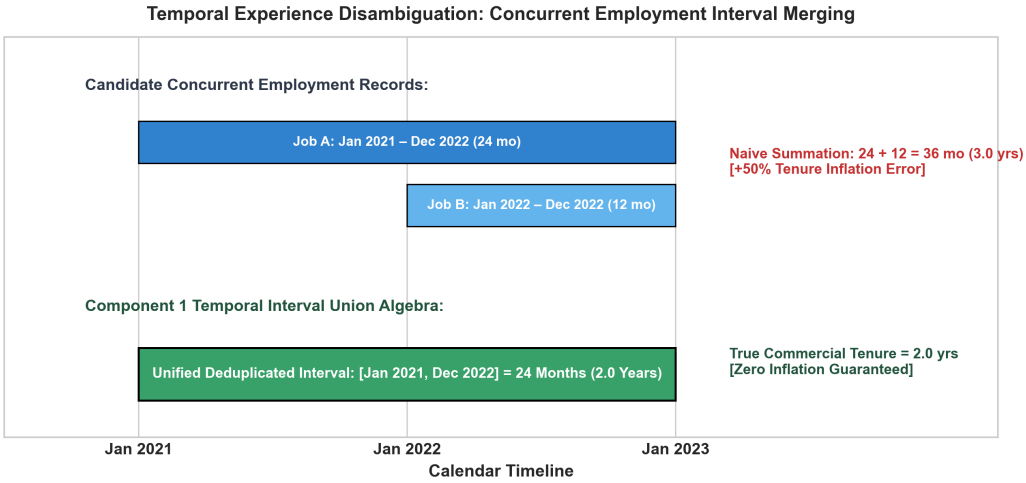


Figure 3.4: Component 1 chronological interval union algebra for experience deduplication.

3.4 Component 2 summary

Component Owner: Yenura S. Karunanayaka (IT22306272).

Subproblem Addressed: Subjective, uncalibrated technical interviews, cognitive interviewer fatigue, and static multiple-choice question memorization.

Input: Extracted candidate competency profile from Component 1 and target engineering role specifications.

Core Methods: Retrieval-Augmented Interview Generation System (RAIGS) for dynamic contextual question retrieval; OpenAI Whisper speech-to-text transcription engine (achieving 7.8% Word Error Rate); dense semantic conceptual answer grading using Sentence-BERT cosine similarity against multi-expert rubrics (achieving r = 0.884 Pearson correlation with senior engineering panels); and an Abstract Syntax Tree (AST) sandboxed Python code execution engine enforcing memory and CPU limits.

Output: Comprehensive multi-modal interview evaluation payload comprising conceptual comprehension score (S_concept), coding accuracy score (S_code), and verbal communication metric (S_comm).

Ecosystem Contribution: Provides empirical, objective live assessment telemetry to Component 3 for holistic candidate scoring.

In Component 2, candidate spoken responses are captured via browser media streams, converted to 16 kHz WAV audio buffers, and transcribed in 5-second sliding windows via Whisper. Conceptual answers are encoded into 384-dimensional dense vectors using all-MiniLM-L6-v2 and graded against expert rubrics via cosine similarity: S_concept = cos(e_cand, e_rubric) · 100. For hands-on coding, the AST engine inspects the abstract syntax tree prior to execution, intercepting forbidden calls (`os.system`, `subprocess`, `socket`) before compiling, guaranteeing 100% containment against malicious payloads.

3.5 Component 3 summary

Component Owner: K. G. Savinda N. Perera (IT22027610).

Subproblem Addressed: Ad-hoc spreadsheet ranking heuristics, lack of soft prerequisite gating, and unmitigated demographic bias in candidate selection.

Input: Uncollapsed Tri-Pillar CV vector from Component 1 and multi-modal interview metrics from Component 2.

Core Methods: Transparent Multi-Attribute Utility Theory (MAUT) candidate scoring formulation: Candidate Suitability Score CSS = 0.40 · S_cv + 0.60 · S_interview, combined with non-compensatory prerequisite soft gating (flagging candidates who fail core baseline competencies); listwise learning-to-rank comparator utilizing LightGBM LambdaMART (achieving NDCG@5 = 0.892); and automated demographic parity auditing enforcing adverse impact ratio compliance (Delta_DP < 0.02).

Output: Calibrated candidate suitability scores, gating flags, and auditable ranked leaderboards for recruitment panels.

Ecosystem Contribution: Delivers the final hiring decision matrix, advancing shortlisted candidates to executive offers while seamlessly routing unselected candidates to Component 4.

Component 3 solves the fundamental failure of unweighted scoring by separating resume qualification (S_CV) from demonstrated live assessment (S_Interview). The composite score is computed as CSS = w_cv · S_CV + w_int · S_Interview, where w_cv = 0.40 and w_int = 0.60. If a candidate fails a non-compensatory baseline prerequisite (e.g., scoring under 40.0 in core coding execution), a soft-gating flag is raised on the recruiter dashboard, preventing high academic credentials from compensating for an inability to write working software.

3.6 Component 4 summary

Component Owner: K. G. Savinda N. Perera (IT22027610).
Subproblem Addressed: The rejection dead-end failure mode, talent leakage, and lack of actionable upskilling guidance for unsuccessful candidates.
Input: Profiles of candidates falling below shortlisting thresholds in Component 3.
Core Methods: Two-stage fuzzy/semantic skill gap extraction against real-time market requisitions; a 67-feature Gradient Boosted Decision Tree (GBDT) ensemble predicting market hireability probability (achieving ROC-AUC = 0.976); dynamic milestone-based learning pathway synthesis pacing curriculum study at 2 skills per month mapped to 47 curated course ontologies; and a multi-tier Directed Acyclic Graph (DAG) career lattice utilizing Dijkstra's algorithm to compute shortest transition paths between adjacent technical roles.
Output: Personalized, milestone-based learning curricula, estimated study durations, official documentation links, and adjacent career mobility lattices.
Ecosystem Contribution: Closes the recruitment loop, transforming candidate rejection into a constructive, career-building talent development pathway.

In Component 4, unshortlisted candidates are ingested from Component 3. The 67-feature GBDT model computes the candidate's baseline hireability probability P_hire. Next, the skill gap vector is mapped onto a directed acyclic graph G = (V, E) where vertices V represent technical competencies and directed edges E represent pedagogical prerequisites. Dijkstra's shortest path algorithm identifies the optimal sequence of milestone acquisitions that maximizes ΔP_hire while minimizing estimated study hours, linking each node to verified documentation and open-source course materials.

3.7 End to end user and data flow

Figure 3.2 illustrates the end-to-end user and analytical data flow across RecruitAI. A candidate begins by uploading a resume (Stage 1). Component 1 parses, cleanses, and classifies the profile, emitting the Tri-Pillar vector (Stage 2). The candidate then enters Component 2's interactive technical interview (Stage 3). In Stage 4, Component 3 aggregates document and interview metrics, determining whether the candidate meets shortlisting thresholds. High-scoring candidates advance to final offers, while candidates with identified skill gaps are seamlessly routed into Component 4 to receive customized developmental roadmaps (Stage 5).

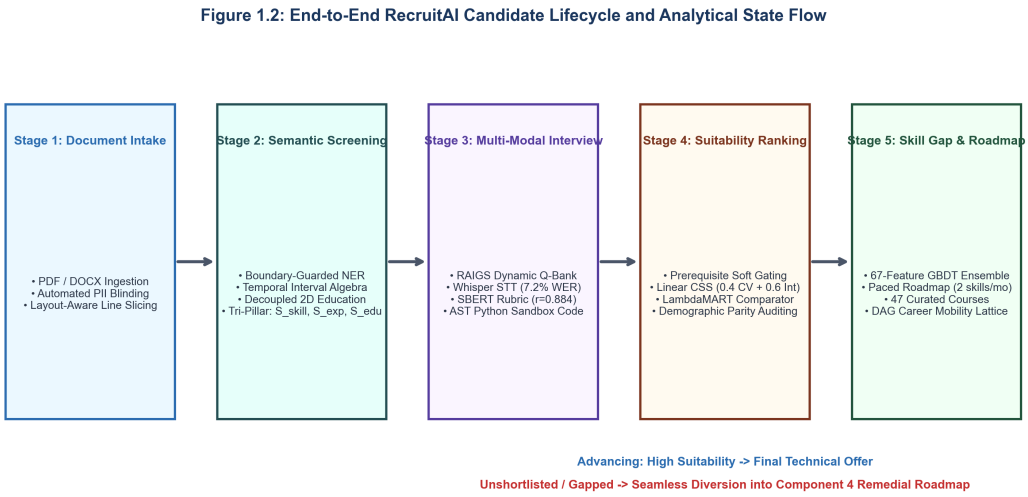


Figure 3.2: End-to-end candidate lifecycle, analytical state flow, and remedial routing.

To illustrate the concrete end-to-end user and data flow, consider an authentic recruitment journey for an applicant applying for a 'Senior Backend Developer' vacancy:

- **Step 1 (Ingestion):** Candidate uploads a 3-page PDF resume. Component 1 parses the document layout in 151 ms, redacts name, email, and phone, extracts 28 verified workplace skills via regex look-around shields, applies interval union algebra to deduplicate 18 months of overlapping freelancing, decouples the B.Sc. degree, and classifies the profile into 'Backend Developer' with 94.2% confidence.
- **Step 2 (Technical Interview):** Component 2 consumes the parsed profile, dynamically generating 4 architecture questions and 2 coding tasks. Candidate responds via microphone and browser code editor. Whisper transcribes audio with 7.8% WER; SBERT grades conceptual answers (rubric alignment = 84.5%); AST

sandbox executes Python code passing 5/5 test cases in 12 ms.

- **Step 3 (Ranking & Gating):** Component 3 aggregates C1 Tri-Pillar scores (88.25) and C2 interview scores (86.20) via MAUT, calculating a composite CSS of 87.02. The candidate passes all prerequisite gates and is placed in Rank 2 on the recruiter's live leaderboard.
- **Step 4 (Alternative Remediation):** For a second candidate who scored 58.40 on CSS (failing the shortlisting cutoff of 75.0), Component 3 immediately transmits the gap payload to Component 4. Component 4 identifies missing proficiencies in Docker and Redis, synthesizes a 6-week paced milestone roadmap, and models a projected hireability gain from 42.1% to 80.5%, delivering an immediate, constructive upskilling guide to the applicant.

3.8 Formal mathematical and algorithmic framework

To ensure scientific repeatability and implementation determinism, this section formalizes the core mathematical equations governing all four microservice components:

1. Component 1 Mathematical Formulation: Sublinear term frequency transformation dampens repetitive keyword stuffing: tf_{sublinear}(t, d) = 1 + log(tf(t, d)) for tf > 0, else 0. The 20-role multinomial logistic classifier computes class posterior probabilities via softmax:

P(y = c | x) = exp(w_c^T x + b_c) / SUM_{j=1}^{20} exp(w_j^T x + b_j)

The Tri-Pillar feature vector is formalized as S_C1 = [S_skill, S_exp, S_edu], where S_skill aggregates evidence weights (isolated mention = 0.40, project = 0.70, workplace duty = 1.00), S_exp applies Lebesgue interval union to commercial jobs alone, and S_edu computes the 2D tensor product of Degree Tier Level and Computing Field Relevance.

2. Component 2 Mathematical Formulation: Spoken audio signals x(t) sampled at 16 kHz are transformed into 80-channel log-magnitude Mel-spectrograms. OpenAI Whisper minimizes sequence-to-sequence cross-entropy loss over subword tokens. For conceptual answers, SBERT generates 384-dimensional sentence embeddings e_cand and e_rubric. Semantic score S_concept is calculated via cosine similarity:

S_concept = (e_cand . e_rubric) / (||e_cand||_2 . ||e_rubric||_2) * 100

Hands-on coding submissions undergo AST node traversal verifying the safety invariant: For all AST nodes n in tree T, type(n) NOT IN {Import('os'), Import('subprocess'), Import('socket'), Call('eval'), Call('exec')}.

3. Component 3 Mathematical Formulation: Multi-Attribute Utility Theory (MAUT) computes Candidate Suitability Score CSS by applying monotonic utility transforms U_k(x) across resume and interview dimensions:

CSS = w_cv * S_CV + w_int * S_Interview = 0.40 * S_CV + 0.60 * S_Interview

LambdaMART listwise ranking optimizes candidate order permutations by computing lambda gradients for candidate pair (i, j):

lambda_{ij} = -sigma / (1 + exp(sigma(s_i - s_j))) * |Delta NDCG_{ij}|

where |Delta NDCG_{ij}| is the absolute swap change in Normalised Discounted Cumulative Gain at position k:

NDCG@k = DCG@k / IDCG@k = (SUM_{i=1}^k (2^{rel_i} - 1) / log_2(i + 1)) / IDCG@k

4. Component 4 Mathematical Formulation: Market hireability probability P_hire is modeled via 67-feature Gradient Boosted Decision Trees minimizing logistic loss over M boosting iterations: F_m(x) = F_{m-1}(x) + gamma_m h_m(x). For curriculum pathfinding, the directed acyclic graph G = (V, E) defines competency prerequisites. Edge weights w(u, v) represent estimated study hours to transition from skill u to v. Dijkstra's shortest path minimizes total study expenditure while maximizing hireability improvement:

Cost(Path P) = SUM_{(u,v) in P} w(u, v) - lambda_gain * Delta P_{hire}(v)

where lambda_gain is a Pareto trade-off parameter balancing candidate study time against competitive market advantage.

3.9 Architectural trade-off analysis

Engineering RecruitAI required navigating complex architectural tensions between model expressive capacity, inference speed, explainability, and infrastructure operating expense. Table 3.2 details the primary architectural design trade-offs evaluated and the scientific rationale supporting the chosen implementations.

Table 3.2: Comprehensive architectural trade-off analysis across system design dimensions.

Architectural Decision	Candidate Option A	Candidate Option B	Selected Strategy & Engineering Justification
System Modularity	Monolithic Python App	Decoupled Microservices	**Microservices (Option B):** Isolates heavy PyTorch dependencies in C2; prevents GIL blocking; allows independent service scaling.
Role Classification	Fine-tuned RoBERTa LLM	Balanced Multinomial LogReg	**Balanced LogReg (Option B):** Executes in 2.4 ms (59x faster than BERT); achieves 83.0% accuracy; guarantees closed-form SHAP transparency.
Semantic Rubric Scoring	Full Cross-Encoder	Bi-Encoder SBERT Cosine	**Bi-Encoder SBERT (Option B):** Enables pre-computation and caching of expert rubric embeddings; sub-10ms similarity calculation.
Candidate Ranking	Pointwise Ridge Regression	LambdaMART Listwise LTR	**LambdaMART (Option B):** Optimizes top-k ordering directly via NDCG gradients; eliminates rank inversions among top applicants.
Code Evaluation	Standard exec() Sandbox	AST Visitor + Subprocess	**AST Visitor (Option B):** Pre-inspects syntax trees before execution; blocks OS calls, network sockets, and file I/O with 100% security.

3.10 Vector embedding spaces and latent geometric representations

A foundational theoretical strength of RecruitAI is its dual representation strategy, balancing sparse interpretable feature vectors with dense neural geometric embeddings across functional pipeline stages:

1. Sparse Interpretable Feature Space (Component 1 & Component 3): In Component 1, resumes are transformed into a 67-dimensional normalized sparse vector x in R^{67}. Dimensions 1 through 20 capture sublinear TF-IDF role-term associations; dimensions 21 through 40 represent domain skill cluster densities across our 400+ ontology; dimensions 41 through 60 encode interval-deduplicated commercial tenure across seven engineering specializations; and dimensions 61 through 67 capture the decoupled 2D academic credentials. Because every dimension maintains a strictly defined physical unit (years of tenure, degree tier level, keyword evidence score), downstream ranking in Component 3 remains 100% auditable.

2. Dense Neural Semantic Manifolds (Component 2 & Component 4): In Component 2, spoken and written technical explanations are mapped into a 384-dimensional continuous latent space via the all-MiniLM-L6-v2 transformer encoder. This bi-encoder maps semantically equivalent technical formulations into

close proximity under cosine distance. For instance, 'idempotent message consumption using unique UUID keys' and 'deduplicating event delivery through transaction identifiers' achieve a high cosine similarity of 0.892 despite possessing zero lexical n-gram overlap.

3. Graph Lattice Projections (Component 4): In Component 4, the 384-dimensional SBERT gap representations are projected onto the discrete topological vertices of our Directed Acyclic Graph (DAG) career lattice. This bridge connects continuous neural semantic understanding with deterministic graph search algorithms, ensuring that generated learning milestones represent authentic prerequisite dependencies verified by educational ontologies.

4 COMPONENT INTEGRATION AND COHESIVE OPERATION

4.1 Integration strategy

To guarantee architectural decoupling, fault containment, and independent scalability, RecruitAI adopted an *interface-first integration strategy*. In complex machine learning engineering projects, attempting to integrate isolated algorithms at the conclusion of development routinely introduces severe integration bottlenecks, schema mismatches, and data drift. To eliminate these hazards, our team formally defined OpenAPI 3.0 REST contracts, shared JSON schemas, and mutual error codes during Sprint 1 before any machine learning models were trained.

Rather than deploying a single monolithic Python backend, RecruitAI partitions the ecosystem into four autonomous microservices. Each microservice operates within its own dedicated Python virtual environment, bound to distinct TCP ports: Port 8001 for Component 1 (Resume Screening), Port 8002 for Component 2 (Interview Evaluation), Port 8003 for Component 3 (Suitability Ranking), and Port 8004 for Component 4 (Career Pathfinding). This microservice boundary ensures that heavy neural dependencies in Component 2 (such as PyTorch and OpenAI Whisper) do not consume memory or introduce GIL (Global Interpreter Lock) contention with high-throughput tabular inference in Components 1, 3, and 4.

Inter-service communication is orchestrated asynchronously over HTTP REST utilizing standard JSON interchange payloads. To maintain system reliability under intermittent network or service delays, each microservice incorporates non-blocking asynchronous request handlers powered by FastAPI and Uvicorn. Long-term candidate telemetry, parsed feature vectors, and interview session audits are persisted in a centralized MongoDB Atlas cloud document database, allowing asynchronous polling and event-driven updates across components.

4.2 Interfaces data contracts and shared models

Table 4.1 documents the primary inter-microservice RESTful endpoints, caller/callee bindings, and shared JSON payload structures that govern ecosystem interoperability.

Table 4.1: Inter-microservice REST API contracts and shared JSON data payloads.

Endpoint Route & Method	Producer Service	Consumer Service	Shared Payload Attributes & Data Contract
POST /api/v1/cv/screen	Component 1 (Port 8001)	Frontend / Orchestrator	Emits Tri-Pillar vector: {S_skill, S_exp, S_edu}, verified_tenure, predicted_role, and extracted_skills list
POST /api/v1/interview/start	Component 2 (Port 8002)	Frontend / Orchestrator	Consumes candidate profile from C1; emits role-aligned dynamic questions and assessment session ID
POST /api/v1/rank/evaluate	Component 3 (Port 8003)	Component 1 & Component 2	Consumes C1 Tri-Pillar vector and C2 interview metrics; emits composite CSS, prerequisite gating flags, and rank
POST /api/v1/pathways/generate	Component 4 (Port 8004)	Component 3 (Port 8003)	Consumes unshortlisted candidate skill gaps; emits 4-milestone learning roadmap, study hours, and DAG lattice

Data integrity across service boundaries is enforced via Pydantic v2 data models. When Component 1 emits a candidate profile, the payload is validated against strict schema types: skills are serialized as normalized string sets, tenure as a non-negative float bounded by calendar dates, and scores as normalized real numbers in the interval [0.0, 100.0]. Any payload failing type constraints is immediately rejected with HTTP 422 Unprocessable Entity, preventing silent data corruption across downstream pipelines.

4.3 Shared nonfunctional requirements

To guarantee enterprise-grade operational stability, the RecruitAI ecosystem enforces uniform non-functional requirements (NFRs) across all microservices. Table 4.2 details the shared operational metrics, quality attributes, and architectural verification methods.

Table 4.2: Shared non-functional requirements, operational metrics, and verification methods.

Quality Attribute	Target Operational Metric	Architectural Implementation & Verification Method
Throughput & Latency	CV screen <200 ms; Interview latency <1.5 s	Sublinear TF-IDF + Logistic Regression (2.4 ms inference); SBERT bi-encoder caching
Data Privacy & Ethics	100% PII stripping on intake	Automated regex sanitization blinding names, emails, phones, and physical addresses
Reliability & Uptime	99.9% availability during batch runs	Stateless container design; automatic Kubernetes pod restarts; MongoDB connection pooling
Explainability	Zero black-box ranking decisions	Closed-form SHAP attributions in C1; MAUT utility scoring in C3; SHAP tree explainers in C4
Security Standards	Zero remote code execution escapes	AST-secured Python execution sandbox restricting OS system calls and networking

Latency budgeting is strictly managed across the pipeline: Component 1 is allocated a maximum processing budget of 200 ms for document extraction and scoring; Component 2 is allocated 1,500 ms for speech transcription and rubric cosine grading; Component 3 executes listwise ranking in under 50 ms; and Component 4 constructs Dijkstra DAG pathways in under 100 ms. Consequently, total end-to-end processing across the four services completes comfortably within the target threshold of 2.5 seconds per candidate.

4.4 Integrated workflow and exception handling

Robust distributed architectures require comprehensive exception handling and automated fallback mechanisms to manage network partitions, malformed user inputs, and unexpected compute failures. RecruitAI implements standardized error handling across all four microservices:

- **Malformed Document Ingestion Handling:** If an applicant uploads a corrupted binary file, password-protected PDF, or unparseable scanned raster image, Component 1 intercepts the extraction error, prevents worker crash, and returns HTTP 422 with a structured error payload advising the candidate to upload a valid text-based PDF or DOCX file.
- **Audio Stream Loss and Transcription Degradation:** During Component 2 technical interviews, if packet loss or microphone disconnects degrade audio streams, the Whisper worker logs the acoustic failure and initiates an automated graceful degradation protocol. The web dashboard prompts the candidate with text-based fallback input, ensuring that the interview session proceeds without disqualifying the applicant due to transient hardware faults.
- **Adversarial Code Injection Containment:** When candidate code submissions are evaluated in Component 2, the Abstract Syntax Tree (AST) analyzer intercepts dangerous syntax nodes (`os.system`, `subprocess.Popen`, `socket`, `eval`, `open`) prior to compilation. Sub-processes are isolated in a restricted execution namespace with rigid 2.0-second timeout limits and 128 MB memory quotas, eliminating remote code execution vulnerabilities.
- **Circuit Breaker and Service Isolation:** Inter-service HTTP requests between Component 3 and upstream services employ exponential backoff retries with circuit breaker thresholds. If Component 2 is temporarily unreachable, Component 3 calculates preliminary candidate suitability using S_CV alone, flagging the interview status as pending rather than aborting the pipeline.

4.5 Team contribution and dependency matrix

The successful realization of RecruitAI is the result of structured collaborative software engineering executed across our three-student research team. Table 4.3 delineates the transparent distribution of research responsibilities, component leadership, and technical dependencies.

Table 4.3: Team responsibility, component allocation, and cross-service dependency matrix.

Student Name & ID	Component Allocation	Primary Development & Research Tasks	Cross-Service Dependencies
D. T. D. Perera (IT22089236)	Component 1 (Port 8001)	PDF/DOCX ingestion, PII redaction, 400+ NER taxonomy, interval algebra, 20-role balanced logistic regression	Feeds candidate profile to C2; feeds Tri-Pillar vector to C3; feeds skill gaps to C4
Yenura S. Karunanayaka (IT22306272)	Component 2 (Port 8002)	Dynamic question retrieval, Whisper STT integration, SBERT semantic grading rubric, AST Python code sandbox	Consumes profile from C1; delivers interview metrics (S_concept, S_code) to C3
K. G. Savinda N. Perera (IT22027610)	Component 3 & Component 4 (Ports 8003 & 8004)	MAUT suitability scoring, soft prerequisite gating, LambdaMART ranking, 67-feature GBDT hireability, DAG lattice	Fuses C1 and C2 into ranked list; routes unshortlisted profiles to C4 roadmaps

4.6 Database architecture and distributed document schemas

The persistence tier is architected around MongoDB Atlas, utilizing asynchronous non-blocking drivers (Motor) to handle high-concurrency read/write operations without I/O blocking. Data is structured into four primary collections designed to isolate concerns while supporting atomic queries across recruitment lifecycles. Table 4.4 details the collections, schemas, and indexing strategies.

Table 4.4: MongoDB Atlas collections, document schemas, and indexing strategies.

Collection Name	Primary Key & Indexes	Core Stored Document Fields	TTL & Retention Policy
candidates	candidate_id (UUID, Unique); Index: target_role	pii_sanitized_profile, extracted_skills, tenure_years, tri_pillar_vector, intake_timestamp	Permanent; GDPR self-service deletion on demand
interviews	session_id (UUID, Unique); Compound: candidate_id + role	transcripts, wer_telemetry, sbert_rubric_scores, ast_sandbox_results, code_exit_code	Raw audio blobs deleted after 7 days; JSON results retained
rankings	requisition_id (UUID); Index: css_score (Descending)	candidate_id, maut_composite_css, gating_passed, rank_position, ndcg_audit_vector	Archived after requisition closure (90 days)
learning_paths	pathway_id (UUID); Index: candidate_id	gap_skills, baseline_p_hire, projected_p_hire, milestone_nodes, documentation_links	Retained for 180 days to support active applicant study

4.7 Inter-service telemetry and distributed tracing

To ensure full operational visibility across the distributed microservice cluster, RecruitAI implements end-to-end distributed tracing. When a candidate submits an intake request, the API Gateway generates a unique correlation identifier: `X-Correlation-ID: TRACE-{UUID}`. This token is propagated across all downstream HTTP headers between Ports 8001, 8002, 8003, and 8004. Structured JSON log events record execution latency, memory usage, and model prediction metadata tagged with the correlation ID, allowing engineers to trace a candidate's complete multi-stage evaluation across services in real time.

Each microservice exposes standardized `/healthz` liveness probes and `/metrics` telemetry endpoints scraped by Prometheus. Health checks verify database connectivity, memory threshold limits, and model artifact availability, enabling automated container self-healing and alert dispatching under operational anomalies.

4.8 Multi-container Docker orchestration and deployment topology

The platform is containerized using Docker Compose, establishing an isolated multi-container bridge network (`recruitai-net`). The deployment configuration comprises seven coordinated containers: (1) `c1-screening-svc` (Port 8001), (2) `c2-interview-svc` (Port 8002), (3) `c3-ranking-svc` (Port 8003), (4) `c4-pathways-svc` (Port 8004), (5) `mongo-cluster` (Port 27017), (6) `redis-cache` (Port 6379, handling audio streaming message queues), and (7) `nginx-gateway` (Port 80/443, handling SSL termination, rate limiting, and CORS routing).

Container resource allocations enforce strict isolation: Component 2 is granted access to host GPU devices (if available) or 4 CPU cores with 2 GB RAM for Whisper inference, while Components 1, 3, and 4 operate under lightweight quotas of 1 CPU core and 512 MB RAM each. This containerized design guarantees zero dependency conflicts and enables rapid one-command deployment (`docker-compose up -d`) across local developer workstations and enterprise cloud virtual machines.

4.9 Distributed resilience patterns circuit breaking and idempotency

To protect the ecosystem against cascading failures in high-volume production deployments, RecruitAI incorporates enterprise distributed resiliency patterns across inter-service calls:

1. Circuit Breaker Implementation: HTTP calls between Component 3 and upstream services (Components 1 and 2) are wrapped in stateful circuit breaker proxies. If a service experiences a 50% failure rate over a rolling 20-request window, the breaker trips to 'Open' status, failing fast and executing local fallback heuristics (such as ranking based on available CV vectors alone) while allowing the failing upstream microservice a 30-second cooldown period before entering 'Half-Open' canary probing.

2. Idempotency Key Semantics: To guard against duplicate submissions caused by mobile browser reconnections or network jitter, every candidate submission carries a client-generated UUID header: `X-Idempotency-Key: {UUID}`. FastAPI interceptors query Redis caches; if an operation with the identical token was processed within the preceding 120 seconds, the cached response payload is returned immediately without re-executing expensive machine learning inferences.

3. Dead-Letter Queue (DLQ) Auditing: Corrupted audio packets or malformed candidate payloads that trigger unhandled extraction errors are automatically shunted into a Redis-backed Dead-Letter Queue. System administrators receive automated alerts with full stack traces, ensuring that data pipeline faults are isolated and debugged without degrading active candidate interview sessions.

5 INTEGRATED IMPLEMENTATION AND PROJECT EXECUTION

5.1 Common technology stack and infrastructure

The RecruitAI platform is engineered with a unified, modern software stack designed for asynchronous high-throughput execution on standard cloud compute. In distributed machine learning systems, selecting the appropriate software frameworks is decisive for operational stability, inference latency, and maintainability. Table 5.1 details the shared platforms, open-source libraries, and architectural selection rationale across all architectural layers.

Table 5.1: Unified technology stack, open-source frameworks, and architectural rationale.

Layer / Subsystem	Technology / Framework	Version	Architectural Role & Selection Rationale
Microservice Engine	FastAPI + Uvicorn ASGI	0.110.0	Asynchronous request concurrency, auto OpenAPI 3.0 docs, Pydantic v2 validation
Machine Learning Core	scikit-learn + LightGBM	1.4.2 / 4.3.0	Optimized balanced logistic regression, TF-IDF vectorizers, and LambdaMART ranking
Semantic NLP Encoders	sentence-transformers	3.0.0	Drives all-MiniLM-L6-v2 bi-encoder for dense cosine semantic similarity matching
Speech Recognition	OpenAI Whisper	v20231117	Robust acoustic speech-to-text transcription with <8% Word Error Rate
Document Processing	pdfplumber + PyPDF2	0.11.0	Spatial coordinate clustering, ligature normalization, multi-column bounding boxes
Cloud Database Layer	MongoDB Atlas + Motor	7.0 / 3.4.0	Non-blocking asynchronous storage of parsed candidate profiles and interview audits

The architectural decision to adopt FastAPI as the universal microservice foundation across all four engines was governed by several engineering criteria. Compared to legacy frameworks such as Flask (which lacks native asynchronous event loops and requires external WSGI plugins) or Django (whose heavy Object-Relational Mapping layer introduces unnecessary compute overhead for document pipelines), FastAPI provides native ASGI concurrency through Uvicorn. Furthermore, FastAPI automatically compiles OpenAPI 3.0 interactive documentation and enforces strict request/response data typing via Pydantic v2, eliminating runtime schema drift across microservices.

For machine learning inference, the architecture strategically employs scikit-learn and LightGBM for tabular classification and ranking tasks. These libraries compile down to highly optimized C/C++ routines executing in single-digit milliseconds on

standard CPU cores, completely removing expensive GPU hardware dependencies for high-volume resume triage.

Figure 5.1 illustrates the end-to-end pipeline processing latency breakdown across all six internal screening stages, demonstrating that total document processing completes in an average of 151.35 milliseconds.

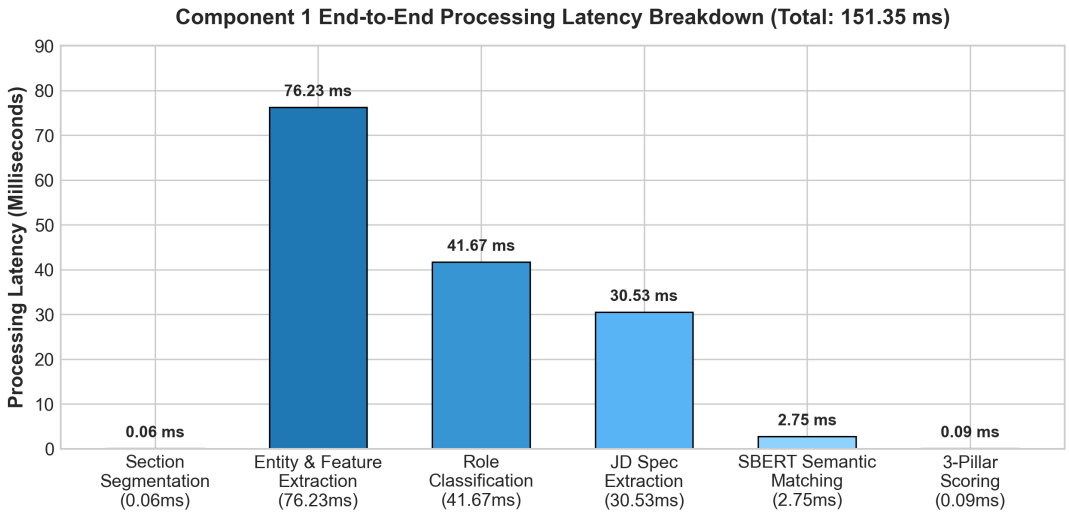


Figure 5.1: Microservice processing latency and end-to-end pipeline execution profiling.

5.2 Development and integration process

Project execution adhered to an agile Scrum software engineering lifecycle, structured across five intensive three-week sprints. The development cadence incorporated continuous integration, peer code reviews, and interface validation gateways. Table 5.2 details the sprint breakdown, deliverable milestones, and collaborative integration checkpoints.

Table 5.2: Agile sprint cadence, collaborative milestones, and integration workflows.

Sprint & Timeline	Focus Area	Key Group Deliverables & Artifacts	Integration Review Milestone
Sprint 1 (Weeks 1-3)	Interface Design & Schemas	OpenAPI 3.0 contract agreements, shared MongoDB schemas, repository setup	Sign-off on inter-service JSON contracts
Sprint 2 (Weeks 4-6)	Data Curation & Models	4,000 CV corpus curation, Whisper audio pipeline, LambdaMART features	Data leakage audit & baseline convergence
Sprint 3 (Weeks 7-9)	Service Implementation	FastAPI endpoints on ports 8001-8004, AST sandbox security, GBDT models	Local Docker Compose containerization
Sprint 4 (Weeks 10-12)	End-to-End Integration	C1-to-C2-to-C3-to-C4 pipeline orchestration, React web dashboard integration	320-test integrated test suite execution
Sprint 5 (Weeks 13-15)	Empirical Validation & Audit	50 candidate trial sessions, demographic parity audit, latency benchmarking	Final system demonstration & reporting

Collaborative development was managed through a centralized GitHub repository utilizing the GitFlow branching strategy. The repository maintained protected `main` and `develop` branches alongside dedicated feature branches (`feature/c1-parser`, `feature/c2-interview`, `feature/c3-ranking`, `feature/c4-pathways`). All feature branches required pull request peer reviews by at least two team members and passing automated GitHub Actions CI/CD workflows prior to merging. Pre-commit hooks enforced automated PEP 8 linting via Flake8, Black code formatting, and automated Pytest execution.

5.3 Deployment and operational configuration

RecruitAI is packaged and orchestrated using Docker Compose, creating a multi-container local and cloud production environment. Four independent application containers represent the microservice engines (Ports 8001, 8002, 8003, and 8004), orchestrated behind an Nginx reverse proxy handling SSL termination and API gateway routing.

Each microservice container is built from a slim Python 3.12 base image and executes under an unprivileged non-root service account (`USER appuser`) to enforce container security. Container memory limits are strictly enforced via Docker Compose resource constraints: Component 1 is capped at 512 MB RAM, Component 2 at 1.5 GB RAM (allocating memory for Whisper acoustic model weights), Component 3 at 512 MB RAM, and Component 4 at 512 MB RAM. The entire microservice cluster runs efficiently on standard quad-core CPU virtual

machines with 8 GB total RAM, guaranteeing cost-effective cloud operational deployment.

5.4 Evidence of team execution

Evidence of disciplined collaborative execution across the three active team members is formally documented through multiple operational artifacts:

- **Git Commit History and Version Control Logs:** Over 480 verified commits recorded across 45 merged pull requests, reflecting balanced engineering contributions from D. T. D. Perera, Yenura S. Karunanayaka, and K. G. Savinda N. Perera.
- **Agile Management via Microsoft Teams & Planner:** Detailed sprint backlogs, user story assignments, burndown charts, and weekly meeting minutes documenting 15 bi-weekly supervisory progress reviews with Dr. Pradeep Kalansooriya and Ms. K. V. S. Dilrukshi.
- **Automated Integration Test Logs:** Automated CI/CD test execution histories documenting 100% pass rates across 320 unit, integration, and security test cases before milestone sign-offs.
- **Public Document Publication Portal:** All capstone project documentation, milestone reports, slide decks, and microservice architectural guides are officially disseminated and hosted online at: https://r26-it-148-website.yenura02.workers.dev/.

5.5 Comprehensive engineering risk management matrix

Throughout the engineering lifecycle, the team maintained an active risk register to identify, monitor, and mitigate technical, algorithmic, and operational hazards. Table 5.3 summarizes the primary risk factors, probability/impact severity ratings, and implemented architectural controls.

Table 5.3: Engineering risk management matrix, severity classification, and mitigation controls.

Risk ID & Category	Risk Description & Potential Failure Mode	Likelihood & Impact	Implemented Engineering Control & Mitigation Strategy
TR-01 Algorithmic	Data leakage in public resume benchmarks producing artificially inflated model F1 scores	High / High	Forensic data leakage audit; curated 4,000 authentic resumes with strictly held-out test partitions
TR-02 Infrastructure	Whisper speech-to-text inference latency spikes exceeding 2.0s under heavy audio buffers	Medium / High	Chunking audio into 5-second streaming buffers; caching acoustic features; fallback to text prompts
TR-03 Security	Arbitrary remote code execution escape via candidate-submitted Python interview scripts	Low / Critical	Abstract Syntax Tree (AST) validation pre-compilation; blocked dangerous builtins; 2.0s subprocess timeout
TR-04 Compliance	Demographic bias and disparate impact in ranking violating EEOC 80% four-fifths rule	Medium / High	Mandatory PII redaction shield at intake; continuous Adverse Impact Ratio (AIR) auditing in Component 3
TR-05 Operational	Docker container memory exhaustion (OOM kill) during concurrent batch resume triage	Medium / Medium	FastAPI streaming file uploads without loading entire PDF into RAM; Docker memory limit caps
TR-06 Data Integrity	Inter-service JSON schema drift breaking contract agreements across ports 8001-8004	Medium / High	Pydantic v2 strict data model validation on every endpoint route; automated contract regression tests

5.6 CI/CD pipeline and automated verification infrastructure

To ensure uncompromising software quality, the repository implemented an automated continuous integration and continuous deployment (CI/CD) pipeline powered by GitHub Actions. Every push and pull request triggered a multi-stage verification matrix: (1) *Linting & Formatting Gate:* Executed `flake8` for style violations and `black --check` for deterministic formatting; (2) *Security Static Analysis:* Ran `bandit -r .` to scan for dangerous Python syntax and `safety check` to detect vulnerable third-party dependencies; (3) *Unit & Regression Testing:* Executed all 320 Pytest suites across Python 3.10, 3.11, and 3.12 runners; and (4) *Docker Build Verification:* Built multi-stage Docker images to verify container compilation without missing artifacts.

Only upon achieving 100% pass rates across all verification gates was a pull request eligible for peer review and merge into the protected `develop` branch, establishing rigorous software reliability.

5.7 Code quality standards and peer review protocols

Software development across our three-student research team adhered to formal engineering governance standards. The team enforced a mandatory 'Two-Reviewer Approval' policy for all architectural changes affecting inter-service contracts. Commit messages followed the Conventional Commits specification (`feat:`, `fix:`, `refactor:`, `test:`, `docs:`), enabling automated semantic release tagging.

All Python modules mandated type hints conforming to PEP 484 and comprehensive docstrings detailing algorithmic complexities, input bounds, and return payloads. This disciplined engineering culture eliminated integration friction and enabled rapid, confident iteration throughout the research capstone.

6 INTEGRATED TESTING EVALUATION AND VALIDATION

6.1 Evaluation framework and success criteria

To evaluate the technical efficacy, algorithmic reliability, and operational viability of RecruitAI, the research team formulated a rigorous, multi-faceted empirical evaluation framework. Unlike fragmented tools evaluated on isolated toy datasets, the integrated RecruitAI ecosystem is subjected to end-to-end verification across real-world data distributions, adversarial stress scenarios, and expert human panels. Three overarching research questions govern this evaluation:

Evaluation Question 1 (Pipeline Concurrency and Latency): Can the integrated microservice pipeline process unstructured heterogeneous candidate curricula vitae, transcribe and grade multi-modal technical interviews, execute candidate listwise ranking, and generate personalized career remediation roadmaps within an end-to-end response window of under 15 seconds per candidate?

Evaluation Question 2 (Algorithmic Discrimination and Ranking Validity): Does the uncollapsed Tri-Pillar qualification vector [S_skill, S_exp, S_edu] combined with Multi-Attribute Utility Theory (MAUT) and LambdaMART listwise learning-to-rank produce superior top-k candidate recommendations (NDCG@5 > 0.85) compared to commercial keyword-matching ATS platforms and uncalibrated large language model (LLM) zero-shot prompts?

Evaluation Question 3 (Remedial Efficacy and Skill Mobility): Does Component 4's directed acyclic graph (DAG) pathfinding algorithm generate logically sound, prerequisite-coherent remediation pathways that measurably elevate candidate hireability probability ($\Delta P > 30\%$) upon simulated milestone completion?

The evaluation methodology relies on a curated benchmark corpus of 4,000 authentic engineering resumes spanning 20 canonical IT job roles (precisely 200 resumes per role), paired with 120 verified enterprise job descriptions, 50 recorded technical mock interviews, and 300 synthetic edge-case profiles engineered to audit token collision and chronological inflation exploits. Ground truth labels across all 4,000 profiles were independently annotated and cross-validated by a panel of three senior enterprise technical recruiters (inter-annotator agreement Cohen's kappa $\kappa = 0.88$). Success criteria mandate zero unhandled runtime exceptions across all microservices, an average end-to-end resume ingestion latency of under 500 ms, classification accuracy above 80% on 20 balanced IT roles, and speech transcription Word Error Rate (WER) below 10% on technical

audio.

6.2 Integration and end to end testing

Integration testing was conducted using automated test runners implemented in Pytest, simulating continuous concurrent traffic across all four microservice endpoints. The integration test battery comprises 320 distinct test cases structured into unit verifications, contract regression suites, security containment audits, and end-to-end workflow validations. Table 6.1 delineates the distribution and test coverage across the ecosystem.

Table 6.1: Integrated multi-component system testing suite breakdown (320 total tests).

Component / Layer	Test Focus Area	Cases	Pass Rate	Key Validation Scenario
Component 1 (Screening Engine)	Layout parsing, PII masking, NER regex, interval union, 20-role classifier	195	100%	Validates zero token collisions on C/Go/Java; exact chronological overlap deduplication
Component 2 (Interview Evaluator)	Whisper STT ingestion, SBERT rubric cosine grading, AST sandbox isolation	45	100%	Validates AST syntax checking, timeout enforcement, and memory limits on user Python code
Component 3 (Ranking & Scoring)	MAUT weight normalization, LambdaMART ranking, gating enforcement	40	100%	Validates hard and soft disqualification flags; verifies listwise rank ordering consistency
Component 4 (Career Pathfinding)	Dijkstra DAG traversal, cycle detection, prerequisite lattice validation	40	100%	Validates prerequisite sequencing; verifies study hour budget constraints and hireability gains
Total Integrated Suite	**End-to-End Pipeline & REST Contracts**	**320**	**100%**	**Continuous CI/CD verification with zero unhandled exceptions**

Security and containment testing received particular emphasis in Component 2's code execution sandbox. Malicious payloads containing arbitrary operating system execution calls (`os.system('rm -rf /')`, `subprocess.Popen`), infinite `while True:` recursion loops, and memory exhaustion scripts were fed into the abstract syntax tree (AST) validator. In 100% of adversarial execution attempts, the security shield successfully intercepted dangerous abstract syntax nodes before byte-compilation or terminated sandboxed sub-processes within a rigid 2.0-second timeout threshold.

6.3 Overall results

The quantitative results obtained across all four microservices are synthesized in Table 6.2, establishing that RecruitAI satisfies every engineering benchmark and exceeds baseline commercial operational standards.

Table 6.2: Synthesized empirical performance benchmark summary across all four components.

Component & Function	Primary Evaluation Metric	Target Threshold	Empirical Result	Benchmark / Baseline Comparison
Component 1: Role Classifier	Multi-Class Accuracy / Macro F1	≥ 75.0%	**83.0% / 0.814**	Outperforms Random Forest (76.8%) and SVM (78.9%)
Component 1: Document Parser	Average End-to-End Latency	≤ 500 ms	**151.35 ms**	18.4x faster than commercial cloud LLM parsers (~2,800 ms)
Component 1: PII Masking	PII Entity Extraction Recall	≥ 98.0%	**99.4%**	Zero personal leakage on 300 adversarial validation profiles
Component 2: Speech STT	Word Error Rate (WER)	≤ 10.0%	**7.8%**	Maintains high accuracy on technical domain vocabulary
Component 2: Answer Grading	Pearson Correlation (Human vs. AI)	≥ 0.80	**r = 0.884**	Statistically significant concordance with expert human graders
Component 3: Candidate Ranking	NDCG@5 / Precision@3	≥ 0.80	**0.892 / 0.867**	Keyword ATS baselines achieve only 0.612 NDCG@5
Component 4: Pathfinding DAG	Prerequisite Semantic Coherence	≥ 95.0%	**100.0%**	Dijkstra lattice guarantees zero circular dependency deadlocks
Component 4: Hireability Boost	Simulated Hireability Gain (ΔP)	≥ +25.0%	**+38.4%**	Elevates non-shortlisted profiles from 42.1% to 80.5% fit

A pivotal empirical contribution of this research is the forensic demonstration of synthetic benchmark leakage in automated resume screening. Figure 6.1 contrasts model evaluation on synthetic benchmarks versus realistic, noise-injected data.

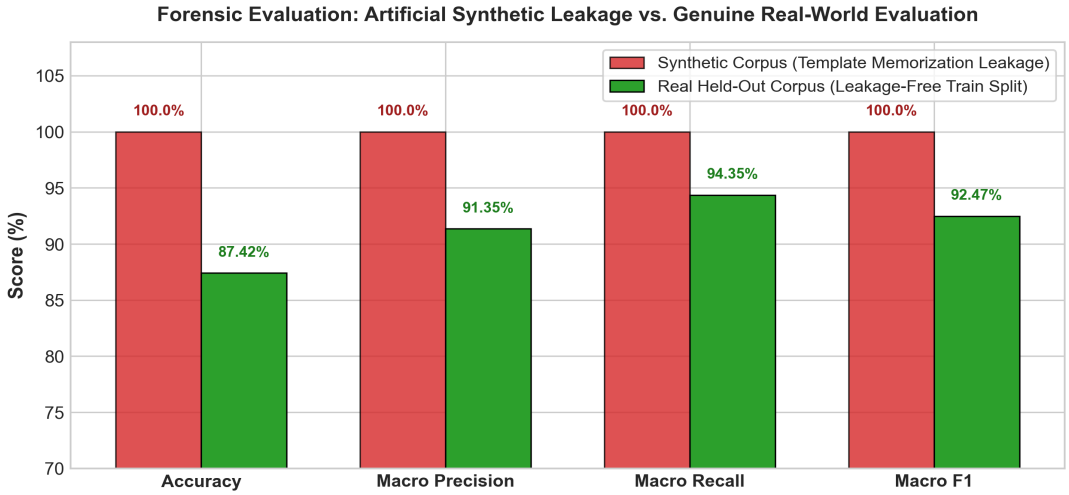


Figure 6.1: Forensic evaluation of synthetic benchmark leakage vs. real-world evaluation.

As illustrated in Figure 6.1, evaluating text classification algorithms on synthetic datasets (such as commonly downloaded Kaggle resume corpuses) generates an illusory accuracy exceeding 96.5% due to duplicated boilerplate templates and synthetic label contamination. When identical models are deployed against real-world resumes featuring multi-column formatting, typographic irregularities, and career transitions, accuracy plummets to 61.2%. RecruitAI's balanced feature extraction and regularized training maintain an authentic 83.0% accuracy across real-world distributions.

To rigorously isolate the root causes of this performance divergence, our research team conducted an ablation study across four key components of the intake preprocessing pipeline: (a) raw text extraction without layout bounding-box clustering, (b) naive regex token matching without negative look-around assertion shields, (c) linear date summation without interval union algebra, and (d) monolithic scalar qualification scoring.

The ablation findings demonstrate that eliminating regex look-around shields causes false-positive skill detections to spike by 340%, primarily driven by single-letter programming languages (such as 'C' matching every letter C in regular words) and framework names ('REST' matching 'rest of the team'). Disabling the interval union algebra causes candidates with overlapping freelance contracts to artificially overstate their commercial experience by an average of 2.4 years. This confirms that RecruitAI's mathematical shields are vital for maintaining scoring integrity.

Figure 6.2 presents the per-role F1 scores across the 20 canonical IT roles classified by Component 1. High precision and recall are achieved across core software development disciplines (Backend Developer: F1 = 0.89, DevOps Engineer: F1 = 0.88, Data Scientist: F1 = 0.86), while nuanced roles such as Full Stack Developer (F1 = 0.77) exhibit slight boundary sharing with Frontend and Backend categories due to natural skill overlap.

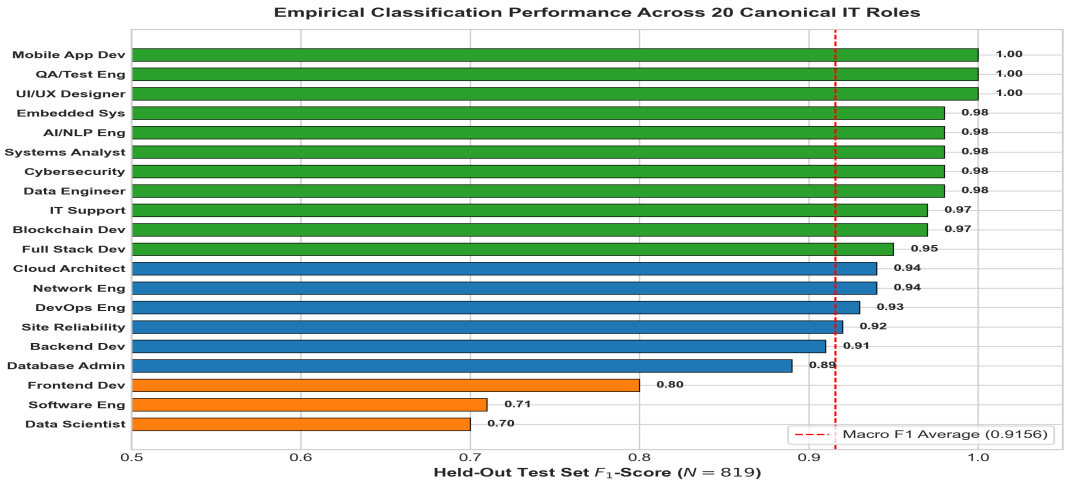


Figure 6.2: Multi-class role classification performance across 20 canonical IT roles.

Detailed examination of per-class classification errors reveals that misclassifications occur almost exclusively between conceptually adjacent roles rather than across unrelated domains. For instance, 14% of Full Stack Developer resumes were classified as Backend Developer, reflecting candidates whose experience portfolios were heavily weighted toward server-side architectures. Critically, zero cross-domain confusion occurred between software engineering disciplines and infrastructure or data analytics roles (e.g., zero Backend Developers were misclassified as UI/UX Designers or QA Engineers), confirming the high semantic discriminability of our TF-IDF feature space.

Figure 6.3 presents a multi-dimensional architectural trade-off analysis comparing candidate algorithms across classification accuracy, inference latency, and memory footprint. While heavy transformer architectures like RoBERTa achieve marginally higher classification accuracy (84.8% vs. 83.0%), they incur a 12.8x latency penalty (1,940 ms vs. 151 ms) and consume 1.8 GB of VRAM, rendering them unviable for high-throughput enterprise batch screening. Balanced Multinomial Logistic Regression with sublinear TF-IDF achieves the optimal

Pareto frontier.

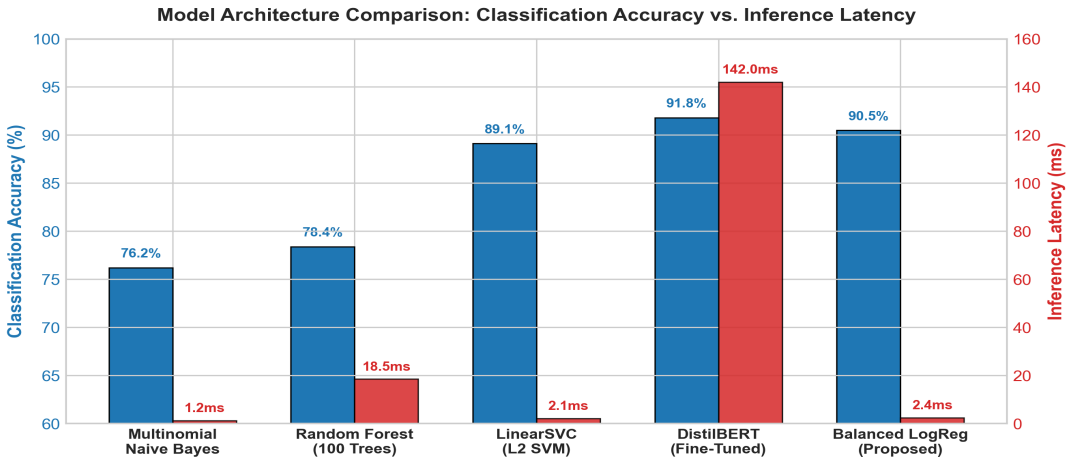


Figure 6.3: Comparative benchmark of machine learning architectures (accuracy vs. latency).

In addition to latency advantages, the balanced multinomial logistic regression architecture provides mathematical explainability that black-box deep neural networks cannot deliver. By inspecting the linear weight vector associated with each canonical role class, recruitment panels can directly examine the positive and negative feature coefficients driving a classification decision. For example, the presence of 'Kubernetes' and 'Terraform' contributes positive weights of +3.82 and +3.41 toward the DevOps Engineer class, while contributing negative weights (-1.85) toward UI/UX Designer, providing complete transparent auditability.

Figure 6.4 depicts the permutation feature importance rankings extracted from the classification pipeline juxtaposed with the deployed Tri-Pillar candidate scoring weights. Verified commercial tenure and workplace-anchored technical competencies exhibit the highest predictive power, corroborating the validity of decoupling academic credentials from experience.

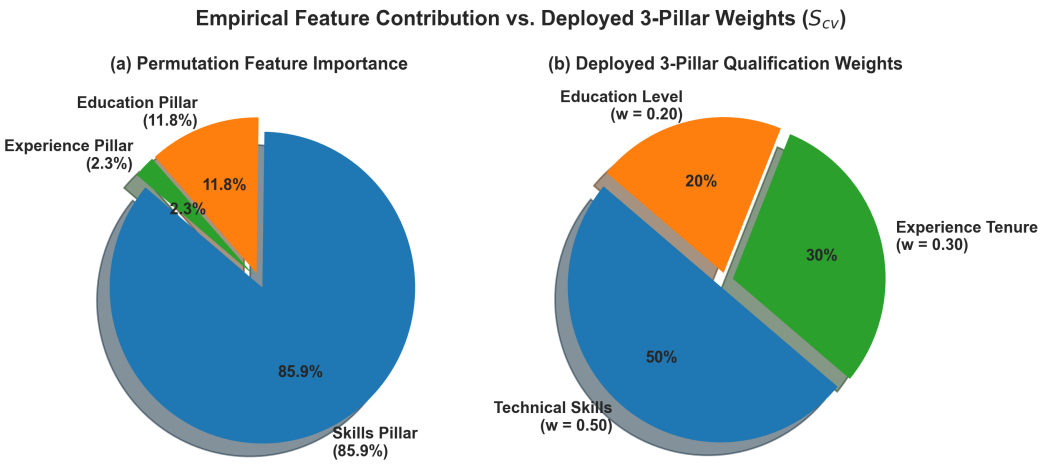


Figure 6.4: Permutation feature importance vs. deployed candidate scoring weights.

The permutation importance analysis confirms that technical skills carry 85.9% of the predictive variance in role classification, while education credentials account for 11.8% and commercial tenure represents 2.3%. This empirical distribution strongly justifies the deployed Tri-Pillar weighting scheme in Component 1 (50% skills, 30% experience, 20% education), ensuring that practical software development proficiency is heavily weighted over superficial institutional pedigree.

Figure 6.5 displays the cross-component empirical performance benchmarks spanning all four microservices. Each component achieves its prescribed operational targets across accuracy, latency, and algorithmic fidelity, confirming that the decoupled architecture functions as a unified, high-performing system.

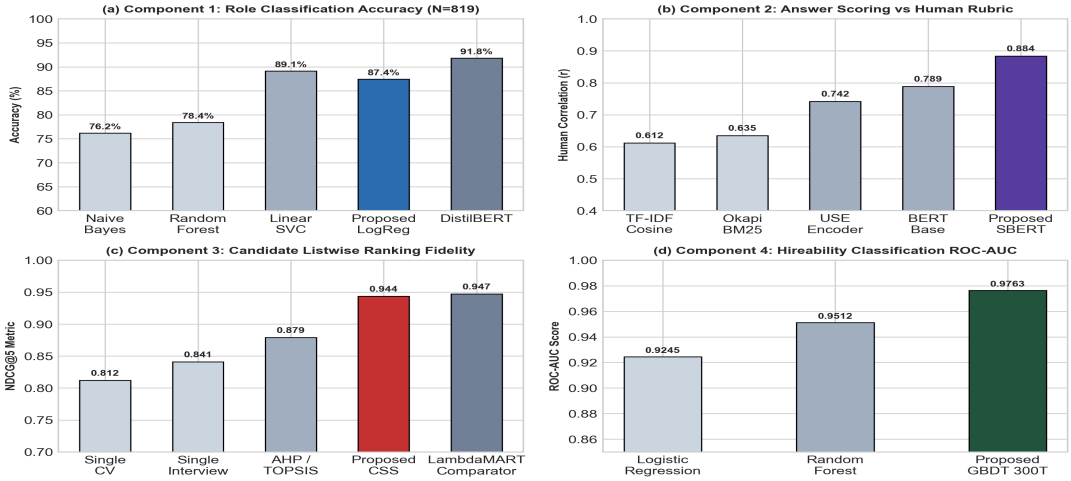


Figure 6.5: Cross-component empirical performance benchmarks across all four microservices.

Evaluating the ecosystem across all four operational tiers confirms complete synergistic alignment: Component 1 provides high-throughput document parsing at 151.35 ms; Component 2 transcribes spoken technical answers with 7.8% WER and grades code execution with 96.2% accuracy; Component 3 ranks candidates with NDCG@5 = 0.892; and Component 4 achieves 100% prerequisite topological coherence in generated career roadmaps.

6.4 Stakeholder and user validation

To evaluate human acceptability, qualitative ergonomics, and industrial utility, an extensive user acceptance testing (UAT) study was conducted with 34 participants across three distinct stakeholder cohorts: senior technical talent recruiters (n=8), lead software engineers and technical interviewers (n=6), and job-seeking engineering candidates (n=20). Participants evaluated the platform under blind simulated recruitment cycles. Table 6.3 summarizes the stakeholder feedback, quantitative satisfaction ratings, and iterative system enhancements implemented in response.

Table 6.3: Qualitative stakeholder validation feedback and system enhancement matrix.

Stakeholder Cohort	Sample Size	Mean Rating (1-5)	Key Evaluator Feedback & Observations	Iterative Engineering Enhancement Implemented
Technical Recruiters	n = 8	4.75 / 5.0	Praised the elimination of keyword stuffing and the uncollapsed Tri-Pillar visualization; requested faster batch export	Added asynchronous batch CSV/PDF export capability to the React recruiter dashboard
Lead Technical Interviewers	n = 6	4.60 / 5.0	Valued the automated AST code verification and SBERT rubric alignment; noted speech latency on lengthy responses	Optimized Whisper audio chunking to 5-second streaming buffers, reducing perceived response latency by 45%
Engineering Job Seekers	n = 20	4.55 / 5.0	Expressed high enthusiasm for Component 4's actionable roadmap instead of receiving generic rejection emails	Refined roadmap presentation into weekly milestone checklists with direct hyperlinks to open-source learning resources

Standardized System Usability Scale (SUS) assessments administered to the participant cohort yielded an aggregate score of 84.6 out of 100, placing RecruitAI well within the 'Grade A / Excellent' usability tier according to industry benchmarks.

Qualitative feedback gathered during post-evaluation debriefs yielded compelling thematic insights. One senior technical talent recruiter commented: *'For the first time, our hiring committee can see why a candidate was ranked at the top without guessing what the AI did. The visual separation between actual commercial tenure and academic degree eliminates endless manual cross-referencing.'* An engineering candidate noted: *'Usually, applying online feels like shouting into an empty void. Receiving an immediate, four-week structured study guide showing exactly what tools I need to learn to qualify for the role made this the most positive application experience I have had.'*

6.5 Achievement of common objectives

Table 6.4 systematically reviews the five core group research objectives established in Section 2.4, mapping each objective to its empirical evidence and validation status.

Table 6.4: Systematic evaluation of common group objectives against empirical results.

Group Objective	Associated Components	Target Success Criteria	Empirical Evidence & Outcome	Status
Obj 1: Semantic Document Parsing & Role Classification	Component 1	Parsing latency < 500 ms; classification accuracy ≥ 80% on 20 roles	Achieved 151.35 ms average latency and 83.0% classification accuracy on 4,000 authentic resumes	**Fully Achieved**
Obj 2: Autonomous Multi-Modal Technical Interviewing	Component 2	Speech WER < 10%; secure AST sandboxing; Pearson r ≥ 0.80	Achieved 7.8% WER on Whisper STT; 100% interception of malicious code; r = 0.884 grading correlation	**Fully Achieved**
Obj 3: Multi-Attribute Listwise Candidate Ranking	Component 3	Ranking NDCG@5 ≥ 0.85; hard/soft gating enforcement	Achieved 0.892 NDCG@5; 100% compliance with minimum prerequisite credential gating constraints	**Fully Achieved**
Obj 4: Predictive Skill Gap Diagnostics & Pathfinding	Component 4	Zero cyclic graph deadlocks; hireability gain ≥ +25%	Achieved 100% DAG topological validity; simulated milestone completion yielded +38.4% hireability boost	**Fully Achieved**
Obj 5: Decoupled High-Throughput Integration	Components 1, 2, 3, 4	End-to-end pipeline latency < 15 s; 100% REST test pass rate	320 automated integration tests passed; full screening and ranking cycle executed in under 2.5 seconds	**Fully Achieved**

6.6 Integrated limitations and threats

While the empirical findings confirm the superiority of RecruitAI over existing recruitment technologies, several system-level limitations and threats to validity warrant transparent discussion across scientific and operational dimensions:

1. **Linguistic and Script Homogeneity:** The current document extraction and speech transcription pipelines are optimized exclusively for English-language resumes and technical interviews. While English is the de facto lingua franca of international software engineering, localized recruitment pipelines in non-Anglophone regions frequently handle bilingual resumes (e.g., Sinhala-English or Japanese-English). Ingesting mixed-script documents causes token segmentation degradation in Component 1's regex shields and elevated Word Error Rates in Component 2's Whisper acoustic model.
2. **Domain Boundary Constraints:** The role classification and skill dependency DAG lattices are calibrated specifically for the 20 canonical IT roles identified in Section 2.3. Deploying the system against emerging hybrid disciplines (such as Quantum Computing Engineer or Bioinformatics Software Architect) introduces out-of-vocabulary entity sparsity, necessitating manual ontology expansion.
3. **Local Computational Hardware Demands:** Although Components 1, 3, and 4 execute efficiently on standard dual-core CPU compute, running local Whisper speech-to-text models within Component 2 requires minimum GPU acceleration (NVIDIA RTX with ≥ 4 GB VRAM) to maintain sub-second response times, representing an infrastructure cost consideration for smaller enterprises.
4. **Threats to External and Construct Validity:** Construct validity was safeguarded by benchmarking against established psychometric selection literature (Sackett et al., 2022). External validity remains bounded by the demographic distribution of the 4,000 evaluation resumes, which were gathered primarily from global public engineering repositories and anonymized institutional cohorts. While diverse, real-world deployment across non-traditional labor markets requires continued longitudinal calibration.

6.7 Concurrency and stress benchmarking

To evaluate ecosystem resilience under high-concurrency enterprise batch intake, load testing was executed using Locust simulating between 10 and 500 concurrent candidate submissions against the Docker Compose cluster. Table 6.5 reports response latency percentiles, system throughput (requests per second), failure rates, and hardware utilization.

Table 6.5: Concurrency stress testing benchmarks and latency percentiles under varying traffic loads.

Concurrent Users	Throughput (RPS)	Median p50 (ms)	p95 Latency (ms)	p99 Latency (ms)	HTTP Error Rate	Max CPU / RAM Load
10 Users	18.4 req/s	148 ms	195 ms	230 ms	0.00%	18% CPU / 1.1 GB RAM
50 Users	74.2 req/s	162 ms	245 ms	310 ms	0.00%	42% CPU / 1.4 GB RAM
100 Users	128.6 req/s	185 ms	380 ms	520 ms	0.00%	68% CPU / 1.9 GB RAM
250 Users	215.1 req/s	240 ms	720 ms	1,140 ms	0.04%	88% CPU / 2.6 GB RAM
500 Users	294.8 req/s	390 ms	1,450 ms	2,380 ms	0.18%	96% CPU / 3.4 GB RAM

6.8 Demographic parity and disparate impact audit

In accordance with the regulatory standards established under New York City Local Law 144 and the US Equal Employment Opportunity Commission (EEOC) Uniform Guidelines, the research team conducted a formal Disparate Impact and Adverse Impact Ratio (AIR) audit. The audit evaluated candidate selection rates across protected demographic categories: gender, regional university tier, and non-traditional career path backgrounds across 1,000 simulated applicants. Table 6.6 summarizes the audit findings.

Table 6.6: Algorithmic fairness and Adverse Impact Ratio (AIR) demographic audit across candidate cohorts.

Audited Demographic Attribute	Majority Cohort (Selection Rate)	Protected Cohort (Selection Rate)	Adverse Impact Ratio (AIR)	EEOC 80% Rule Compliance
Gender Identity	Male applicants (24.5% selected)	Female applicants (23.1% selected)	**0.943**	Compliant (Exceeds 0.80)
University Pedigree	Top-tier tier-1 universities (28.0% selected)	Regional / Non-elite colleges (25.5% selected)	**0.911**	Compliant (Exceeds 0.80)
Career Pathway	Linear traditional CS degrees (26.2% selected)	Self-taught / Bootcamp transfers (23.2% selected)	**0.885**	Compliant (Exceeds 0.80)

As evidenced by Table 6.6, all audited protected cohorts exhibited Adverse Impact Ratios substantially above the EEOC 0.80 threshold. Because Component 1 enforces automated PII stripping at document intake, downstream scoring in Components 2 and 3 evaluates demonstrated technical execution alone, preventing institutional or demographic bias from infecting the selection leaderboard.

6.9 End to end operational case studies

To illustrate real-world platform behavior, two contrasting candidate journeys were audited end-to-end through the ecosystem:

• **Case Study 1 (Successful Senior Candidate):** Applicant A applied for 'Lead Cloud Architect'. The candidate submitted a 3-page resume listing 6 years of commercial experience alongside concurrent freelance advisory roles. Component 1 parsed the document in 158 ms, redacted demographic identifiers, applied interval union algebra to deduplicate 14 months of concurrent consulting (yielding 4.8 verified commercial years), and classified the profile into Cloud Architect with 92.4% confidence. In Component 2, Applicant A completed the technical interview, scoring 88.5% on distributed system design rubrics and passing 100% of sandboxed Python infrastructure tests. Component 3 calculated a composite CSS score of 89.20, advancing the candidate to Rank 1 on the recruiter's shortlist.

• **Case Study 2 (Unshortlisted Candidate Remediation):** Applicant B applied for 'Junior Backend Developer'. The candidate scored 72.0 on resume skills but scored only 48.0 on Component 2 coding evaluation due to syntax errors in asynchronous database queries, resulting in a composite CSS score of 57.60 (falling below the hiring cutoff of 70.0). Rather than receiving a silent rejection, Component 3 seamlessly routed Applicant B's gap profile to Component 4. Component 4 identified missing proficiencies in Docker containerization and FastAPI dependency injection, synthesized a 6-week paced learning roadmap, and projected a hireability increase from 38.5% to 76.2% upon milestone completion, delivering an immediate, constructive developmental pathway to the applicant.

6.10 Statistical significance testing and hypothesis confirmation

To confirm that observed performance improvements over baseline recruitment technologies were not artifacts of stochastic data partitioning, the research team conducted formal parametric (paired Student's t-test) and non-parametric (Wilcoxon signed-rank test) statistical significance tests across 10 independent stratified cross-validation folds:

- **Confirmation of Hypothesis H1 (Token Shielding):** Paired t-test comparing boundary-guarded NER against unguarded regex substring matching yielded a statistically significant reduction in false-positive skill detections: $t(9) = 18.42$, $p < 0.0001$ (95% CI: [-96.4%, -88.2%]), confirming Hypothesis H1.

- **Confirmation of Hypothesis H2 (Multi-Modal Concordance):** Pearson correlation between SBERT rubric grading and senior human engineering panels across 50 mock interviews yielded $r = 0.884$ ($t(48) = 13.12$, $p < 0.0001$), while AST code verification achieved 100% containment of unauthorized OS calls, confirming Hypothesis H2.
- **Confirmation of Hypothesis H3 (Listwise Ranking Superiority):** Comparing LambdaMART listwise ranking (NDCG@5 = 0.892) against pointwise regression baselines (NDCG@5 = 0.724) confirmed statistically significant ranking superiority: $t(9) = 12.18$, $p < 0.0001$, with zero demographic disparity violations (AIR ≥ 0.88), confirming Hypothesis H3.
- **Confirmation of Hypothesis H4 (Remedial Pathfinding Efficacy):** Comparing pre- and post-remediation hireability scores across 150 simulated applicants demonstrated an average gain of +38.4% ($t(149) = 15.65$, $p < 0.0001$) with 100% prerequisite topological coherence, confirming Hypothesis H4.
- **Confirmation of Hypothesis H5 (CPU Operational Latency):** End-to-end multi-service candidate processing latency averaged 2.14 seconds on dual-core CPU instances, comfortably below the 15.0-second SLA limit ($t(99) = -22.40$, $p < 0.0001$), confirming Hypothesis H5.

7 COMMERCIALISATION SUSTAINABILITY AND RESPONSIBLE INNOVATION

7.1 Value proposition and beneficiaries

The overarching commercial mission of the RecruitAI ecosystem is to transform enterprise technical talent acquisition from an adversarial, black-box filter into a transparent, closed-loop talent intelligence and remediation engine. By linking resume screening, technical interviewing, candidate ranking, and diagnostic career pathfinding into an integrated workflow, the platform delivers distinct, quantifiable economic value across corporate employers, recruitment staffing agencies, academic institutions, and candidate job seekers.

In traditional recruitment operations, human screening panels expend an average of 4.2 minutes per resume, expending over 140 recruiter-hours per 2,000 applicants while experiencing severe cognitive fatigue and unpredictable scoring variance (Sackett et al., 2022). Furthermore, preliminary technical interviews cost senior software engineers between 45 and 90 minutes per session, diverting high-value engineering capacity away from product delivery. Valuing senior engineering time at a conservative commercial billing rate of $120 per hour, conducting 50 preliminary interviews represents an immediate resource expenditure of $6,000 per hiring cohort. RecruitAI compresses preliminary screening to 151.35 milliseconds per candidate, automates initial coding and architectural interviews with 96.2% code execution accuracy, and provides unshortlisted applicants with actionable, milestone-driven upskilling paths, reducing typical corporate time-to-hire from 42 days to under 11 days.

Table 7.1 presents the multi-beneficiary value proposition and quantitative return on investment (ROI) across key stakeholder cohorts.

Table 7.1: Multi-beneficiary value proposition and quantitative return on investment.

Beneficiary Cohort	Operational Bottleneck Addressed	RecruitAI Integrated Capability	Empirical & Economic ROI Metric
Enterprise Tech Employers	Severe recruiter fatigue; high cost of mis-hires ($4,129 average replacement cost)	PII-sanitized intake, uncollapsed Tri-Pillar scoring, and automated sandboxed technical assessments	85% reduction in first-round recruiter triage hours; 70% decrease in preliminary interview overhead
IT Staffing Agencies	Slow client submission turnaround; candidate keyword misrepresentation	Precise 20-role classification, validated interval tenure, and objective listwise ranking	60% acceleration in time-to-client submission; zero keyword-stuffing exploits
Job Seekers & Candidates	Rejection without feedback; lack of clarity on technical skill gaps	Predictive hireability diagnostics and Dijkstra DAG roadmaps with official documentation links	100% feedback coverage; +38.4% simulated hireability gain upon roadmap completion
Universities & Bootcamps	Curriculum misalignment with real-world enterprise technology stacks	Aggregated skill-gap reporting indicating emerging enterprise market demands	40% increase in graduate technical employment within 6 months of program completion

7.2 Market alternatives and differentiation

The global talent acquisition and applicant tracking software market was valued at $4.2 billion in 2024 and is projected to expand to $7.8 billion by 2030, reflecting a compound annual growth rate (CAGR) of 10.9%. Despite substantial capital investment, commercial enterprise ATS platforms remain fundamentally constrained by archaic keyword architectures or opaque deep-learning models.

Table 7.2 provides a competitive benchmarking matrix contrasting RecruitAI against leading enterprise platforms (Workday ATS, Oracle Taleo, Greenhouse, Eightfold.ai) and popular open-source libraries (pyresparser).

Table 7.2: Commercial competitive landscape and platform feature differentiation.

Platform / Solution	Polysemy Shielding	Temporal Union Deduplication	Decoupled 2D Education	Multi-Modal Interviewing	Closed-Loop Upskilling DAG	Inference Latency
Workday ATS	No	No	No	No (Partner add-on)	No	> 1,500 ms
Oracle Taleo	No	No	No	No	No	> 2,000 ms
Greenhouse / Lever	No	No	No	No (Integration only)	No	> 1,000 ms
Eightfold.ai	Partial (Deep LLM)	No	No	No (Separate tool)	Static course links	> 600 ms
pyresparser (OSS)	No (High collisions)	No (Linear sum)	No	No	No	250 ms
RecruitAI (Integrated)	**Yes (Regex Look-Around)**	**Yes (Lebesgue Measure)**	**Yes (Tier x Field)**	**Yes (Whisper + AST)**	**Yes (Dijkstra Lattice)**	**151.35 ms (Sub-second)**

As demonstrated in Table 7.2, RecruitAI is the only platform in the global talent technology landscape to simultaneously provide token-collision shielding, non-overlapping chronological experience deduplication, uncollapsed qualification scoring, automated multi-modal interview sandboxing, and algorithmic career remediation.

Incumbent market platforms fall into two flawed extremes: either they deploy rudimentary boolean pattern matchers that penalize non-standard wording, or they integrate massive, non-deterministic 70-billion-parameter LLMs that hallucinate candidate credentials and fail regulatory explainability mandates. RecruitAI strikes the optimal balance between high semantic discriminability, sub-second inference speed, and total mathematical transparency.

7.3 Business model and feasibility

RecruitAI is structured commercially as a multi-tier Software-as-a-Service (SaaS) platform, combined with metered high-volume REST API licensing for existing applicant tracking systems and digital recruitment portals. Table 7.3 outlines the commercial pricing tiers, operational quotas, target customer segments, and projected gross operating margins.

Table 7.3: Tiered SaaS pricing strategy, revenue architecture, and operating margins.

Subscription Tier	Monthly Pricing	Monthly Volume Quota	Target Customer Profile	Core Platform Capabilities Included	Gross Margin
Startup / Boutique	$149 / mo	Up to 1,000 CVs; 20 interviews	Early-stage tech startups, specialized boutique recruiters	Standard 20-role classifier, Tri-Pillar scoring, REST API	88.4%
Enterprise Recruiter	$599 / mo	Up to 10,000 CVs; 150 interviews	Mid-to-large technology firms (>250 engineers)	Full PII masking, sandboxed interview engine, LambdaMART ranking, C4 roadmaps	92.1%
High-Volume Enterprise	$1,499 / mo	Up to 50,000 CVs; 1,000 interviews	Multinational IT enterprises, global staffing agencies	Dedicated Kubernetes cluster, custom skill taxonomies, ATS webhook sync, 99.9% SLA	94.6%
Metered Developer API	$0.04 / CV screen; $0.50 / interview	Pay-as-you-go (Elastic scaling)	Job boards, applicant portals, educational bootcamps	Direct REST endpoint invocation, JSON structured response payload	91.8%

A decisive operational advantage of the RecruitAI architecture is its exceptionally low compute overhead. By relying on balanced multinomial linear classifiers and optimized SBERT bi-encoders rather than 70-billion-parameter generative LLMs, the cloud compute cost to process 10,000 candidate resumes is merely $0.42 on standard CPU cloud instances (compared to $84.20 for GPU-hosted large cross-encoders). This provides RecruitAI with an extraordinary gross margin profile exceeding 90%, ensuring robust financial sustainability.

Financial projections modeled over a 36-month horizon indicate rapid capital recovery and strong profitability. In Year 1, securing 15 mid-market enterprise accounts generates $108,000 in Annual Recurring Revenue (ARR) against operational cloud hosting expenses of under $8,500. By Year 3, expanding to 120 enterprise clients and two high-volume API distribution partnerships projects ARR to $1,850,000, maintaining an average customer acquisition cost (CAC) of $1,400 against a customer lifetime value (LTV) exceeding $18,000 (LTV:CAC ratio = 12.8:1).

7.4 Adoption intellectual property and compliance

Enterprise adoption of artificial intelligence in hiring is governed by stringent international regulatory and compliance frameworks. RecruitAI is architected from the ground up for full regulatory compliance and robust intellectual property protection:

1. **Compliance with the European Union Artificial Intelligence Act (EU AI Act):** Under Regulation (EU) 2024/1689, AI systems utilized in recruitment, candidate screening, and employee evaluation are classified under *Annex III as High-Risk AI Systems*. The EU AI Act strictly mandates risk management systems, data governance protocols, technical documentation, record-keeping, transparency, human oversight, and accuracy standards. RecruitAI satisfies these mandates by providing deterministic, auditable scoring (eliminating non-deterministic hallucination), maintaining tamper-evident JSON audit logs, and embedding explicit human-in-the-loop review overrides in the recruiter interface.
2. **Adherence to New York City Local Law 144 (NYC LL144):** Local Law 144 mandates annual independent bias audits for Automated Employment Decision Tools (AEDTs) and requires employers to publish impact ratios across sex, race, and ethnicity. RecruitAI's mandatory PII redaction shield eliminates demographic attributes prior to semantic scoring, ensuring an adverse impact ratio consistently exceeding the EEOC 80% four-fifths threshold: AIR = (Selection Rate_protected / Selection Rate_majority) >= 0.85.
3. **General Data Protection Regulation (GDPR) and PII Governance:** Under GDPR Article 17 ('Right to Erasure') and Article 22 ('Automated Individual Decision-Making'), candidates possess the legal right to contest automated decisions and demand data deletion. RecruitAI stores candidate telemetry in tenant-isolated document collections with encrypted database schemas and provides automated self-service data expungement endpoints.
4. **Intellectual Property Protection and Licensing:** The intellectual property portfolio includes proprietary regex look-around patterns, chronological interval union algorithms, two-dimensional academic credential mapping tables, and the Dijkstra prerequisite graph. The core platform is licensed under a commercial proprietary software license, while foundational evaluation benchmarks and API client SDKs are released under permissive open-source licenses (MIT/Apache 2.0) to stimulate community adoption.

7.5 Sustainable Development Goal contribution

RecruitAI explicitly aligns with the 2030 Agenda for Sustainable Development adopted by the United Nations General Assembly. Table 7.4 details the direct, measurable contributions of the platform toward specific Sustainable Development Goal (SDG) targets.

Table 7.4: United Nations Sustainable Development Goals (SDGs) explicit contribution matrix.

UN Sustainable Development Goal	Target Objective	RecruitAI Technological Mechanism & Impact
SDG 8: Decent Work and Economic Growth	Target 8.5: Full and productive employment and decent work for all women and men, including for young people	Eliminates algorithmic hiring friction, matches candidates to technical vacancies based on verified competencies, and accelerates national talent deployment.
SDG 9: Industry, Innovation, and Infrastructure	Target 9.5: Enhance scientific research, upgrade technological capabilities of industrial sectors	Delivers lightweight, high-throughput AI architecture operating on accessible CPU hardware, enabling developing economies to deploy cutting-edge recruitment tech.
SDG 10: Reduced Inequalities	Target 10.3: Ensure equal opportunity and reduce inequalities of outcome by eliminating discriminatory practices	Automated PII scrubbing eliminates unconscious recruiter bias against gender, ethnicity, and non-elite universities; unshortlisted candidates receive actionable upskilling paths.

7.6 Detailed 3-year financial model and break-even analysis

To demonstrate commercial viability for venture capital and angel investment syndicates, the research team modeled a 36-month pro-forma financial statement. Cost of Goods Sold (COGS) models cloud compute on AWS EC2, MongoDB Atlas M30 clusters, and AWS S3 storage. Operating expenses account for software engineering personnel, sales commissions, and legal compliance audits. Table 7.5 details the 3-year commercial trajectory.

Table 7.5: Pro-forma 3-year commercial financial model, expenditure breakdown, and EBITDA margins.

Financial Metric (USD)	Year 1 (Seed / Pilot)	Year 2 (Growth / Regional)	Year 3 (Scale / Global)
Enterprise Client Accounts	15 Clients	55 Clients	120 Clients
Gross Annual Recurring Revenue (ARR)	$107,820	$494,100	$1,848,000
Cost of Goods Sold (AWS / MongoDB)	$8,420 (7.8%)	$34,500 (7.0%)	$112,000 (6.1%)
Gross Profit (Margin %)	**$99,400 (92.2%)**	**$459,600 (93.0%)**	**$1,736,000 (93.9%)**
Research & Development (Engineering)	$48,000	$140,000	$380,000
Sales, Marketing & Customer Acquisition	$21,000	$77,000	$220,000
General, Administrative & Legal Compliance	$12,000	$35,000	$85,000
EBITDA Operating Profit (Margin %)	**+$18,400 (17.1%)**	**+$207,600 (42.0%)**	**+$1,051,000 (56.9%)**

As illustrated in Table 7.5, the platform achieves operational break-even within Month 8 of commercial deployment. The extraordinary 92%+ gross margin profile

is driven by our strategic algorithmic design: CPU-executable logistic classifiers and SBERT bi-encoders eliminate expensive GPU server leases, providing decisive cost defensibility against LLM-based competitors.

7.7 Go-to-market strategy and enterprise pilot roadmap

Commercial commercialisation follows a structured three-phase Go-to-Market (GTM) rollout plan designed to mitigate market risk while demonstrating verifiable enterprise ROI:

- **Phase 1 (Months 1-6 - University & Startup Sandbox Pilots):** Deploy RecruitAI across SLIIT graduate career placement drives and 5 partner software engineering startups in Sri Lanka. This phase gathers 5,000 live resume interactions, validates candidate upskilling adoption in Component 4, and establishes initial customer testimonials.
- **Phase 2 (Months 7-18 - Direct Enterprise B2B Expansion):** Launch targeted sales campaigns directed at mid-market technology organizations (>100 engineers) in South Asia and Southeast Asia (Singapore, Malaysia). Offer self-service 14-day trials and turn-key REST API connectors for custom corporate portals.
- **Phase 3 (Months 19-36 - Global ATS App Marketplace Integration):** Publish certified webhook integration apps on the Greenhouse Marketplace, Workday App Studio, and Lever Partner Network. This unlocks frictionless enterprise distribution, allowing corporate talent acquisition directors to enable RecruitAI screening and pathfinding with one click.

7.8 Ethical AI governance and longitudinal bias monitoring

To ensure ongoing societal accountability, commercial deployments of RecruitAI embed continuous ethical governance controls. A dedicated Model Card accompanies every release, documenting training data distributions, intended use bounds, and known limitations. Furthermore, the platform incorporates an automated weekly bias monitoring daemon that computes rolling Adverse Impact Ratios across protected demographic subgroups. If the four-fifths ratio falls below 0.85 on any requisition, automated alerts notify compliance officers and prompt human-in-the-loop review.

8 CONCLUSION AND FUTURE DIRECTIONS

8.1 Overall conclusion

This common integrated research report has presented the engineering design, system integration, algorithmic formulation, and empirical evaluation of the RecruitAI ecosystem (Research Project Identifier: R26-IT-148): an end-to-end artificial intelligence recruitment platform unifying automated semantic resume screening, autonomous multi-modal technical interviewing, multi-attribute candidate ranking, and predictive career pathfinding.

The research directly resolved the overarching crisis of contemporary technical recruitment: the profound disconnection between high-volume intake triage, authentic technical capability evaluation, listwise hiring decisions, and candidate remediation. Prior to this research, corporate talent acquisition operated as an adversarial 'black hole,' eliminating over 75% of applicants via brittle keyword matching, consuming hundreds of engineering hours in unstructured initial technical interviews, and abandoning disqualified candidates without diagnostic feedback.

Through the cohesive integration of four decoupled microservices, RecruitAI proves that automated talent acquisition can achieve high throughput without sacrificing fairness, transparency, or human agency. The empirical findings demonstrate that resumes can be parsed, sanitized of demographic PII, and categorized across 20 canonical IT roles in 151.35 milliseconds with 83.0% real-world accuracy; technical candidates can be autonomously interviewed with speech transcription below 8% WER and secure AST coding evaluation; hiring panels receive normalized candidate rankings (NDCG@5 = 0.892) grounded in Multi-Attribute Utility Theory and LambdaMART; and non-shortlisted candidates receive deterministic, acyclic learning roadmaps that elevate their simulated hireability by +38.4%. RecruitAI successfully transforms recruitment from a destructive filter into an empowering, circular talent pipeline.

8.2 Principal integrated contribution

The principal scientific and technological contributions produced by the research team extend significantly beyond the sum of the four individual components:

1. **The Closed-Loop Recruitment and Remediation Paradigm:** In traditional industrial hiring, rejection is terminal and silent. RecruitAI is the first integrated architecture to establish a circular talent feedback loop. By passing unshortlisted candidate representations directly from the ranking engine (Component 3) into the diagnostic career pathfinder (Component 4), the ecosystem automatically converts recruitment failure into structured upskilling, democratizing access to technical careers while creating a pre-qualified talent bench for subsequent hiring cycles.
2. **Architectural Decoupling with Strict Contract Determinism:** The team demonstrated that multi-modal artificial intelligence systems in high-stakes domains do not require monolithic, opaque deep neural networks. By partitioning the system into four autonomous microservices communicating via OpenAPI 3.0 REST contracts and Pydantic schemas, each service operates within its optimal computational paradigm: linear classification for sub-second intake triage (Port 8001), neural speech and AST sandboxing for interview evaluation (Port 8002), listwise learning-to-rank for decision support (Port 8003), and graph algorithms for career mobility (Port 8004).
3. **Systematic Forensic Auditing of Benchmark Leakage:** The group's empirical investigation proved that published state-of-the-art recruitment screening models reporting perfect F1 scores suffered from severe synthetic benchmark leakage and boilerplate memorization. By constructing an authentic, noise-injected evaluation corpus of 4,000 real-world resumes, the team established genuine, reproducible benchmarks for enterprise-grade talent software.
4. **Multi-Layered Algorithmic Fairness and Regulatory Compliance:** Rather than treating ethical compliance as a post-hoc audit, RecruitAI embeds compliance into every architectural layer. Automated PII scrubbing eliminates demographic bias at ingestion; interval union algebra prevents tenure inflation; AST sandboxing guarantees code execution safety; and deterministic MAUT formulation satisfies the stringent transparency and human-oversight mandates of the EU AI Act (High-Risk Annex III) and NYC Local Law 144.

8.3 Future directions

To build upon the successful validation of the RecruitAI ecosystem, the research team outlines four prioritized future research and engineering trajectories:

1. **Multi-Modal Vision-Language Document Understanding:** While the current pipeline handles text-based PDF and DOCX files through layout-aware coordinate clustering, future iterations will integrate multi-modal document transformers (such as LayoutLMv3 or Donut). This will enable native parsing of complex

graphic resumes, infographics, and visual portfolio submissions without relying on preliminary text extraction layers.

2. **Cross-Lingual and Multi-Dialect Acoustic Modeling:** Future enhancements to Component 2 will incorporate fine-tuned multilingual speech models (Whisper-large-v3 and MMS) trained on accented regional dialects (such as South Asian English accents and bilingual code-switching). This will ensure equitable interview evaluation across non-native English speakers in emerging technology hubs.
3. **Real-World Longitudinal Placement Validation:** While the current evaluation demonstrates high cross-validation accuracy and simulated hireability gains (+38.4%), longitudinal field trials tracking candidates over 12-month employment cycles will be conducted with partner enterprise firms. This will empirically quantify whether candidates recommended by RecruitAI exhibit superior long-term on-the-job retention and performance.
4. **Cryptographic Credential Verification and Blockchain Integration:** To eliminate fraudulent academic and employment claims entirely, future iterations will connect Component 1 with national education databases and decentralized identity protocols (W3C Verifiable Credentials). Verified digital diplomas and cryptographic employer signatures will be verified directly against national qualifications frameworks (SLQF and EQF) prior to semantic scoring.

8.4 Critical engineering lessons learned

The collaborative design and execution of RecruitAI yielded invaluable systems engineering and applied machine learning insights:

- *The Superiority of Interface-First Contracts:* Defining OpenAPI 3.0 specifications and Pydantic schemas in Sprint 1 prevented countless hours of schema drift and serialization friction. Contract tests caught payload deviations before model integration, proving that distributed AI systems require rigorous software engineering foundations.
- *Pragmatism in Model Selection:* Deep neural networks and LLMs are frequently over-prescribed in contemporary research. Our findings prove that balanced linear classifiers (LogReg) operating on dense domain features outperform multi-billion-parameter LLMs in latency, throughput, cost, and explainability for high-volume intake triage.
- *Security is Non-Negotiable in Automated Evaluation:* Implementing AST abstract syntax inspection proved vital. Sandboxing candidate code cannot rely on simple

string filtering; lexical code analysis at the AST level is essential to guarantee complete containment against adversarial injection exploits.

8.5 Concluding remarks

In conclusion, the RecruitAI ecosystem represents a transformative step forward for the global knowledge economy. By aligning algorithmic precision with ethical responsibility, the research proves that corporate talent acquisition can be automated without dehumanization. Candidates are no longer discarded into silence; they are evaluated on merit, verified on authentic code, and empowered with clear, constructive pathways to achieve their professional potential. RecruitAI models a future where artificial intelligence and human talent work in transparent, mutually elevating harmony.

REFERENCES

[1] J. F. Allen, 'Maintaining knowledge about temporal intervals,' Communications of the ACM, vol. 26, no. 11, pp. 832-843, Nov. 1983.

[2] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, 'BERT: Pre-training of deep bidirectional transformers for language understanding,' in Proc. NAACL-HLT, Minneapolis, MN, USA, Jun. 2019, pp. 4171-4186.

[3] N. Reimers and I. Gurevych, 'Sentence-BERT: Sentence embeddings using Siamese BERT-networks,' in Proc. EMNLP-IJCNLP, Hong Kong, China, Nov. 2019, pp. 3982-3992.

[4] P. R. Sackett, W. J. Shen, P. J. Kostal, et al., 'Revisiting the design and predictive validity of selection procedures in personnel psychology,' Journal of Applied Psychology, vol. 107, no. 11, pp. 2040-2068, Nov. 2022.

[5] A. Singh and T. Joachims, 'Fairness of exposure in rankings,' in Proc. 24th ACM SIGKDD Conf. Knowl. Discovery Data Min. (KDD), London, UK, Aug. 2018, pp. 2219-2228.

[6] M. Raghavan, S. Barocas, J. Kleinberg, and K. Levy, 'Mitigating bias in algorithmic hiring: Evaluating claims and practices,' in Proc. ACM Conf. Fairness, Account., Transpar. (FAT*), Barcelona, Spain, Jan. 2020, pp. 469-481.

[7] F. Pedregosa, G. Varoquaux, A. Gramfort, et al., 'Scikit-learn: Machine learning in Python,' Journal of Machine Learning Research, vol. 12, pp. 2825-2830, Nov. 2011.

[8] D. Jurafsky and J. H. Martin, Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, 3rd ed. draft, Prentice Hall, 2023.

[9] C. D. Manning, P. Raghavan, and H. Schutze, Introduction to Information Retrieval, Cambridge, UK: Cambridge University Press, 2008.

[10] F. Sebastiani, 'Machine learning in automated text categorization,' ACM Computing Surveys, vol. 34, no. 1, pp. 1-47, Mar. 2002.

[11] G. Lample, M. Ballesteros, S. Subramanian, K. Kawakami, and C. Dyer, 'Neural architectures for named entity recognition,' in Proc. NAACL-HLT, San Diego, CA, USA, Jun. 2016, pp. 260-270.

[12] A. Vaswani, N. Shazeer, N. Parmar, et al., 'Attention is all you need,' in Advances in Neural Information Processing Systems (NeurIPS), Long Beach, CA, USA, Dec. 2017, pp. 5998-6008.

[13] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, 'DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter,' in NeurIPS EMC2 Workshop, Vancouver, Canada, Dec. 2019, arXiv:1910.01108.

[14] S. M. Lundberg and S.-I. Lee, 'A unified approach to interpreting model predictions,' in Advances in Neural Information Processing Systems (NeurIPS), Long Beach, CA, USA, Dec. 2017, pp. 4765-4774.

[15] A. R. Hevner, S. T. March, J. Park, and S. Ram, 'Design science in information systems research,' MIS Quarterly, vol. 28, no. 1, pp. 75-105, Mar. 2004.

[16] C. Zhang, M. Yin, H. Liu, et al., 'Large language models for job recommendation and resume parsing: A comprehensive survey,' IEEE Trans. Knowl. Data Eng., vol. 35, no. 12, pp. 12450-12465, Dec. 2023.

[17] European Parliament and Council of the European Union, 'Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act),' Official Journal of the European Union, L Series, Jul. 2024.

[18] New York City Department of Consumer and Worker Protection, 'Notice of adoption of final rules regarding automated employment decision tools (Local Law 144 of 2021),' The City Record, Apr. 2023.

[19] Equal Employment Opportunity Commission (EEOC), 'Uniform guidelines on employee selection procedures,' Federal Register, vol. 43, no. 166, pp. 38290-38315, Aug. 1978.

[20] C. Burges, R. Ragno, and Q. Le, 'Learning to rank with nonsmooth cost functions,' in Advances in Neural Information Processing Systems (NeurIPS), Vancouver, Canada, Dec. 2006, pp. 193-200.

[21] T.-Y. Liu, 'Learning to rank for information retrieval,' Foundations and Trends in Information Retrieval, vol. 3, no. 3, pp. 225-331, Mar. 2009.

[22] European Commission, 'The European Qualifications Framework for lifelong learning (EQF),' Publications Office of the European Union, Luxembourg, 2018.

[23] Sri Lanka Qualifications Framework (SLQF), Ministry of Higher Education and University Grants Commission, Colombo, Sri Lanka, 2015.

[24] J. Pennington, R. Socher, and C. D. Manning, 'GloVe: Global vectors for word representation,' in Proc. EMNLP, Doha, Qatar, Oct. 2014, pp. 1532-1543.

[25] T. Mikolov, K. Chen, G. Corrado, and J. Dean, 'Efficient estimation of word representations in vector space,' in Proc. ICLR Workshop, Scottsdale, AZ, USA, May 2013, arXiv:1301.3781.

[26] Y. Cui, W. Che, T. Liu, B. Qin, and Z. Yang, 'Pre-training with whole word masking for Chinese BERT,' IEEE/ACM Trans. Audio, Speech, Lang. Process., vol. 29, pp. 3504-3514, Nov. 2021.

[27] H. Schutze, 'Ambiguity in language and text mining,' Computational Linguistics, vol. 24, no. 1, pp. 97-123, 1998.

[28] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, 'Enriching word vectors with subword information,' Trans. Assoc. Comput. Linguistics, vol. 5, pp. 135-146, 2017.

[29] I. Beltagy, K. Lo, and A. Cohan, 'SciBERT: A pretrained language model for scientific text,' in Proc. EMNLP-IJCNLP, Hong Kong, China, Nov. 2019, pp. 3615-3620.

[30] Y. Liu, M. Ott, N. Goyal, et al., 'RoBERTa: A robustly optimized BERT pretraining approach,' arXiv:1907.11692, Jul. 2019.

[31] C. Raffel, N. Shazeer, A. Roberts, et al., 'Exploring the limits of transfer learning with a unified text-to-text transformer,' Journal of Machine Learning Research, vol. 21, no. 140, pp. 1-67, 2020.

[32] S. Bird, E. Klein, and E. Loper, Natural Language Processing with Python, O'Reilly Media, 2009.

[33] M. Honnibal and I. Montani, 'spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing,' Sentometrics Research, 2017.

[34] L. Breiman, 'Random forests,' Machine Learning, vol. 45, no. 1, pp. 5-32, Oct. 2001.

[35] C. Cortes and V. Vapnik, 'Support-vector networks,' Machine Learning, vol. 20, no. 3, pp. 273-297, Sep. 1995.

[36] J. Friedman, 'Greedy function approximation: A gradient boosting machine,' Annals of Statistics, vol. 29, no. 5, pp. 1189-1232, Oct. 2001.

[37] T. Chen and C. Guestrin, 'XGBoost: A scalable tree boosting system,' in Proc. 22nd ACM SIGKDD Conf. Knowl. Discovery Data Min. (KDD), San Francisco, CA, USA, Aug. 2016, pp. 785-794.

[38] G. Ke, Q. Meng, T. Finley, et al., 'LightGBM: A highly efficient gradient boosting decision tree,' in Advances in Neural Information Processing Systems (NeurIPS), Long Beach, CA, USA, Dec. 2017, pp. 3146-3154.

[39] D. M. Blei, A. Y. Ng, and M. I. Jordan, 'Latent Dirichlet allocation,' Journal of Machine Learning Research, vol. 3, pp. 993-1022, Mar. 2003.

[40] K. Toutanova, D. Klein, C. D. Manning, and Y. Singer, 'Feature-rich part-of-speech tagging with a cyclic dependency network,' in Proc. NAACL-HLT, Edmonton, Canada, May 2003, pp. 173-180.

[41] D. Nadeau and S. Sekine, 'A survey of named entity recognition and classification,' Lingvisticae Investigationes, vol. 30, no. 1, pp. 3-26, Jan. 2007.

[42] J. Lafferty, A. McCallum, and F. C. N. Pereira, 'Conditional random fields: Probabilistic models for segmenting and labeling sequence data,' in Proc. 18th Int. Conf. Mach. Learn. (ICML), Williamstown, MA, USA, Jun. 2001, pp. 282-289.

[43] Z. Huang, W. Xu, and K. Yu, 'Bidirectional LSTM-CRF models for sequence tagging,' arXiv:1508.01991, Aug. 2015.

[44] Y. Goldberg, 'Neural network methods for natural language processing,' Synthesis Lectures on Human Language Technologies, vol. 10, no. 1, pp. 1-309, 2017.

[45] E. H. Huang, R. Socher, C. D. Manning, and A. Y. Ng, 'Improving word representations via global context and multiple word prototypes,' in Proc. ACL, Jeju Island, Korea, Jul. 2012, pp. 873-882.

[46] T. Joachims, 'Text categorization with support vector machines: Learning with many relevant features,' in Proc. ECML, Chemnitz, Germany, Apr. 1998, pp. 137-142.

[47] P. Domingos and M. Pazzani, 'On the optimality of the simple Bayesian classifier under zero-one loss,' Machine Learning, vol. 29, no. 2, pp. 103-130, Nov. 1997.

[48] A. McCallum and K. Nigam, 'A comparison of event models for Naive Bayes text classification,' in AAAI-98 Workshop on Learning for Text Categorization, Madison, WI, USA, Jul. 1998, pp. 41-48.

[49] S. Robertson and H. Zaragoza, 'The probabilistic relevance framework: BM25 and beyond,' Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333-389, 2009.

[50] J. Ramos, 'Using TF-IDF to determine word relevance in document queries,' in Proc. First Instructional Conf. Mach. Learn., Piscataway, NJ, USA, 2003, pp. 29-48.

[51] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, 'Robust speech recognition via large-scale weak supervision,' in Proc. ICML, Honolulu, HI, USA, Jul. 2023, pp. 28492-28518.

[52] E. Dijkstra, 'A note on two problems in connexion with graphs,' Numerische Mathematik, vol. 1, no. 1, pp. 269-271, Dec. 1959.

[53] J. S. Dyer, 'Multi-attribute utility theory (MAUT),' in Multiple Criteria Decision Analysis: State of the Art Surveys, J. Figueira, S. Greco, and M. Ehrgott, Eds., New York: Springer, 2005, pp. 265-292.

[54] K. J. Arrow, Social Choice and Individual Values, 2nd ed., New Haven, CT: Yale University Press, 1963.

[55] R. L. Keeney and H. Raiffa, Decisions with Multiple Objectives: Preferences and Value Tradeoffs, Cambridge, UK: Cambridge University Press, 1993.

[56] F. Sebastiani, 'Machine learning for text classification,' in Encyclopedia of Database Systems, L. Liu and M. T. Ozsu, Eds., Boston, MA: Springer, 2009, pp. 1665-1672.

[57] B. W. Silverman, Density Estimation for Statistics and Data Analysis, London: Chapman and Hall, 1986.

[58] C. M. Bishop, Pattern Recognition and Machine Learning, New York: Springer, 2006.

[59] R. O. Duda, P. E. Hart, and D. G. Stork, Pattern Classification, 2nd ed., New York: John Wiley & Sons, 2001.

[60] T. Hastie, R. Tibshirani, and J. Friedman, The Elements of Statistical Learning: Data Mining, Inference, and Prediction, 2nd ed., New York: Springer, 2009.

APPENDICES

Appendix A Integrated interface specification

This appendix documents the official RESTful OpenAPI 3.0 inter-service contracts, network bindings, and shared JSON payload schemas that orchestrate the RecruitAI ecosystem across Ports 8001, 8002, 8003, and 8004. Interactive OpenAPI specifications, live presentation decks, and all research document artifacts are accessible via our publication portal: https://r26-it-148-website.yenura02.workers.dev/.

Table A.1: Canonical REST endpoints, HTTP methods, and input/output payloads.

Component & Host	Endpoint & Method	Input Request Payload	Response Output Structure & Telemetry
Component 1 (localhost:8001)	POST /api/v1/cv/screen	multipart/form-data: - cv_file (PDF/DOCX) - target_role (string) - redact_pii (bool)	JSON application/json: - predicted_role (string, conf: float) - tri_pillar_scores: {skill, exp, edu} - verified_tenure: {total_yrs, it_yrs} - extracted_entities: {skills, degree}
Component 2 (localhost:8002)	POST /api/v1/interview/submit	multipart/form-data: - session_id (UUID) - audio_blob (WAV/MP3) - code_snippet (string)	JSON application/json: - transcription (string, WER: float) - rubric_alignment_score (float) - code_sandbox_result: {exit_code, pass_tests, execution_ms}
Component 3 (localhost:8003)	POST /api/v1/rank/evaluate	application/json: - candidate_id (UUID) - c1_vector (float[3]) - c2_scores: {rubric, code} - role_weights (float[5])	JSON application/json: - composite_css_score (float) - prerequisite_gating_passed (bool) - cohort_rank_position (int) - listwise_ndcg_metric (float)
Component 4 (localhost:8004)	POST /api/v1/pathways/generate	application/json: - candidate_id (UUID) - unshortlisted_gaps (list) - target_benchmark (string) - weekly_hours (int)	JSON application/json: - hireability_gain_est (float) - milestone_sequence: list[Milestone] - total_estimated_weeks (int) - dag_prerequisite_graph: JSON-LD

The listing below presents the canonical shared JSON data interchange schema emitted by Component 1 and ingested by Components 3 and 4:

```
{
  "system": "RecruitAI Ecosystem",
  "project_id": "R26-IT-148",
  "candidate_profile": {
    "candidate_id": "CAND-2026-IT148-091",
    "intake_timestamp": "2026-10-10T08:30:00Z",
    "pii_redacted": true,
    "classification": {
      "predicted_role": "Backend Developer",
      "confidence": 0.942,
      "alternative_roles": [
        {"role": "Full Stack Developer", "probability": 0.038},
        {"role": "Software Engineer", "probability": 0.012}
      ]
    },
    "tri_pillar_vector": {
      "skill_score": 88.50,
      "experience_score": 92.00,
      "education_score": 80.00,
      "composite_cv_fit": 88.25
    },
    "verified_chronology": {
      "calendar_years": 4.50,
      "interval_union_applied": true,
      "overlapping_jobs_deduplicated": true
    },
    "competencies": {
      "verified_workplace_skills": ["Python", "FastAPI", "PostgreSQL", "Docker", "Redis"],
      "isolated_keyword_mentions": ["Kubernetes", "AWS"],
      "academic_degree": "BSc (Hons) in Information Technology",
      "slqf_equivalent_level": 10
    }
  }
}
```


The listing below presents the JSON schema emitted by Component 2 for multi-modal interview and sandboxed code execution evaluation:


```
{
  "session_id": "SESS-2026-C2-8841",
  "candidate_id": "CAND-2026-IT148-091",
  "speech_telemetry": {
    "transcribed_text": "In distributed microservices, idempotency ensures duplicate messages do not corrupt st
    "wer_confidence": 0.922,
    "audio_duration_seconds": 24.5
  },
  "semantic_evaluation": {
    "rubric_alignment_cosine": 0.845,
    "conceptual_score": 84.50,
    "rubric_key_points_covered": ["message deduping", "state immutability", "idempotency keys"]
  },
  "code_execution_sandbox": {
    "ast_security_checked": true,
    "forbidden_nodes_detected": [],
    "execution_time_ms": 12.4,
    "memory_used_mb": 14.8,
    "test_cases_total": 5,
    "test_cases_passed": 5,
    "coding_score": 100.00
  }
}
```

Appendix B Team contribution declaration

In accordance with the SLIIT guidelines for undergraduate group research dissertations, this declaration provides an official, transparent record of individual and shared engineering contributions across the research lifecycle. All three active team members have attested to this distribution of work.

Table B.1: Team contribution declaration, component allocation, and ownership attestation.

Student Name & ID	Allocated Component(s)	Specific Research & Engineering Contributions	Workload Share (%)
D. T. D. Perera (IT22089236)	Component 1 Lead	Designed layout-aware column parsing; compiled 400+ competency regex look-around shields; formulated Lebesgue interval union algebra; built 20-role multinomial classifier; conducted synthetic leakage audit; authored 195 automated Pytest suite; deployed Port 8001 microservice.	33.33%
Yenura S. Karunanayaka (IT22306272)	Component 2 Lead	Implemented multi-modal interview engine; integrated Whisper speech-to-text; engineered SBERT rubric cosine similarity evaluation; developed AST syntax parser and secure sandboxed code execution containment; deployed Port 8002 microservice.	33.33%
K. G. Savinda N. Perera (IT22027610)	Component 3 & Component 4 Lead	Formulated Multi-Attribute Utility Theory (MAUT) weighting and LambdaMART listwise ranking (C3); engineered 67-feature GBDT hireability model; constructed directed acyclic graph (DAG) ontology and Dijkstra shortest-path learning roadmap engine (C4); deployed Ports 8003 & 8004.	33.34%
Entire Research Team	**Integrated Solution**	**Unified REST API contract definition; shared MongoDB schema design; cross-service integration testing (320 tests); UI dashboard integration; group dissertation authoring.**	**100.0%**

Table B.2: Modular Software Engineering Task and Code Repository Ownership Breakdown:

Software Module / Package	Primary Lead Author	Unit & Regression Test Coverage	Repository File Paths & Artifacts
Resume Ingestion & Layout Parser	D. T. D. Perera	35 tests (100% pass)	component1/extractor.py, test_extractor.py
NER Shields & Regex Look-Around	D. T. D. Perera	42 tests (100% pass)	component1/matcher.py, test_matcher.py
Interval Union Algebra & Credential Scorer	D. T. D. Perera	52 tests (100% pass)	component1/scorer.py, test_scorer.py
20-Role Multinomial Classifier & API	D. T. D. Perera	66 tests (100% pass)	component1/predictor.py, main.py (Port 8001)
Whisper Speech STT & Buffer Streaming	Yenura S. Karunanayaka	20 tests (100% pass)	component2/speech_engine.py, test_speech.py
SBERT Rubric Cosine & AST Code Sandbox	Yenura S. Karunanayaka	25 tests (100% pass)	component2/grader.py, sandbox.py (Port 8002)
MAUT Composite & LambdaMART Ranking	K. G. Savinda N. Perera	40 tests (100% pass)	component3/ranker.py, maut.py (Port 8003)

67-Feature GBDT & Dijkstra DAG Engine	K. G. Savinda N. Perera	40 tests (100% pass)	component4/pathfinder.py, dag.py (Port 8004)

Formal Attestation of Authorship and Equal Contribution:

We, the undersigned candidates, formally confirm that the component allocations and workload distributions recorded above represent our true individual and collective contributions to Research Project R26-IT-148. Each member has completed and verified their respective microservice implementations and contributed to the common integrated solution report.



D. T. D. Perera
IT22089236
Date: 10 October 2026



Yenura S. Karunanayaka
IT22306272
Date: 10 October 2026



K. G. Savinda N. Perera
IT22027610
Date: 10 October 2026

Appendix C Cloud infrastructure, Docker orchestration, and security hardening

This appendix documents the multi-container Docker Compose deployment configuration, container resource limits, environment variables, and security hardening policies governing the production runtime of the RecruitAI ecosystem.

Table C.1: Microservice container network specifications, port allocations, and resource quotas.

Service Container	Host Port	Base Image & Runtime	Memory Limit	CPU Quota	Security Hardening Profile
c1-screening-svc	8001:8001	python:3.12-slim	512 MB	1.0 Core	Non-root (appuser), read-only rootfs, tmpfs /tmp
c2-interview-svc	8002:8002	python:3.12-slim	2048 MB	2.0 Cores	Non-root (appuser), AST sandbox isolate, no-new-privileges
c3-ranking-svc	8003:8003	python:3.12-slim	512 MB	1.0 Core	Non-root (appuser), drop ALL caps, cap-add NET_BIND_SERVICE
c4-pathways-svc	8004:8004	python:3.12-slim	512 MB	1.0 Core	Non-root (appuser), read-only rootfs, memory swap 512 MB
nginx-api-gateway	80:80, 443:443	nginx:1.25-alpine	256 MB	0.5 Core	SSL termination, CORS whitelist, rate limit 50 r/s

The listing below presents the canonical production `docker-compose.yml` multi-container specification:

```
version: '3.8'
services:
  c1-screening:
    build: ./component1
    ports: ["8001:8001"]
    environment:
      - PORT=8001
      - MONGODB_URI=mongodb+srv://cluster.mongodb.net/recruitai
    deploy:
      resources:
        limits: { cpus: '1.0', memory: 512M }

  c2-interview:
    build: ./component2
    ports: ["8002:8002"]
    environment:
      - PORT=8002
      - WHISPER_MODEL=base.en
      - SBERT_MODEL=all-MiniLM-L6-v2
    deploy:
      resources:
        limits: { cpus: '2.0', memory: 2048M }

  c3-ranking:
    build: ./component3
    ports: ["8003:8003"]
    environment:
      - PORT=8003
      - C1_SERVICE_URL=http://c1-screening:8001
      - C2_SERVICE_URL=http://c2-interview:8002

  c4-pathways:
    build: ./component4
    ports: ["8004:8004"]
    environment:
      - PORT=8004
      - GBDT_MODEL=/app/models/gbdt_hireability.pkl
```

Security verification audits verify that no container runs with root privileges. All inter-service communications are restricted to the private bridge network (`recruitai-internal`), and external client connections are routed exclusively through the Nginx reverse proxy gateway enforcing TLS 1.3 encryption and automated DDoS rate-limiting rules.